sz笔记

操作系统（Operating System）

计算机系统概述

> 整理自《王道》

操作系统基本概念

- 操作系统定义：

> 操作系统是指控制和管理整个计算机系统的**硬件与软件资源**，合理地组织、调度计算机的**工作与资源的分配**，进而为用户和其他软件提供方便**接口与环境**的**程序集合**。操作系统是计算机系统中**最基本的系统软件**。

- 操作系统的**基本特征**：

> - 并发：
> - **并发**是指两个或多个事件**在同一时间间隔内发生**。
> - 操作系统的并发性是指计算机系统中同时存在多个运行的程序，因此它具有处理和调度多个程序同时执行的能力。
> - **引入进程的目的是使程序能并发执行**。
>
> > 注意同一时间间隔（并发）和**同一时刻**（并行）的区别：
> > - 并发：
> > - 在多道程序环境下，一段时间内，**宏观上**有多道程序在同时执行。
> > - 而在每个时刻，单处理机环境下实际仅能有一道程序执行。
> > - 因此微观上这些程序仍是**分时交替执行**的。操作系统的并发性是通过**分时**得以实现的。
> > - 并行：
> > - 并行性是指系统具有同时进行运算或操作的特性，在同一时刻能完成两种或两种以上的工作。
> > - 并行性需要有相关硬件的支持，如多**流水线或多处理机**硬件环境。
>
> - 共享：

共享是指系统中的资源可供内存中多个**并发**执行的进程共同使用，分为两类：

- 互斥共享方式：

  仅当进程 A 访问完并释放该资源后，才允许另一个进程 B 对该资源进行访问。即在一段时间内只允许一个进程访问该资源，这种资源称为**临界资源或**独占资源**。

- 同时访问方式：

  宏观上在一段时间内允许多个进程「同时」访问某些资源，**微观上**「轮流」（交替）访问。**同时访问的典型资源是磁盘设备。**

> 操作系统**最基本的特征**是**并发和共享**，两者互为存在条件。

- 虚拟：

  虚拟是指把一个物理上的实体变为若干逻辑上的对应物，有以下应用：

- **时分复用**技术：

  虚拟处理器技术是通过多道程序设计技术，采用让多道程序**并发**执行的方法，来分时使用一个处理器的。

- **空分复用**技术：

  虚拟存储器技术将一台机器的物理存储器变为虚拟存储器，以便从逻辑上扩充存储器的容量。

- 异步：

  多道程序环境允许多个程序**并发**执行，但由于资源有限（资源竞争），进程的执行以**不可预知**的速度向前推进，这就是**进程的异步性**。

- 操作系统作为**计算机系统资源**的管理者的功能：

  - 处理机管理：

    - 在多道程序环境下，**处理机的分配和运行都以进程（或线程）为基本单位**，因而对处理机的管理可归结为对进程的管理。
    - 并发是指在计算机内同时运行多个进程，因此**进程何时创建、何时撤销、如何管理、如何避免冲突、合理共享**就是进程管理的最主要的任务。
    - 进程管理的主要功能包括**进程控制、进程同步、进程通信、死锁处理、处理机调度**等。

  - 存储器管理：

        - 存储器管理是为了给多道程序的运行提供良好的环境，方便用户使用及提高内存的利用率。
        - 主要包括**内存分配与回收、地址映射、内存保护与共享和内存扩充**等功能。
    - 文件管理：
        - 计算机中的信息都是以文件的形式存在的，操作系统中负责文件管理的部分称为文件系统。
        - 文件管理包括**文件存储空间的管理、目录管理及文件读写管理和保护**等，但不关心文件具体内容，如：源代码、编译器。
    - 设备管理：
        - 设备管理的主要任务是完成用户的 I/O 请求，方便用户使用各种设备，并提高设备的利用率。
        - 主要包括**缓冲管理、设备分配、设备处理和虚拟设备**等功能。
- 操作系统作为用户与计算机硬件系统之间的**接口**（了解）：
    - 命令接口：

        用户利用这些操作命令来组织和控制作业的执行，分为两类：

        - 联机命令接口：

            「雇主」说一句话，「工人」做一件事，并做出反馈，这就强调了交互性。

        - 脱机命令接口：

            「雇主」把要「工人」做 的事写在清单上，「工人」按照清单命令逐条完成这些事。用户不能直接干预作业的运行。
    - 程序接口：

        程序接口由一组**系统调用（也称广义指令）**组成。用户通过在程序中使用这些**系统调用**来请求操作系统为其提供服务，如使用各种外部设备、申请分配和回收内存及其他各种要求。

        > 当前最为流行的是图形用户界面（GUI），即图形接口。GUI 最终是通过调用**程序接口**实现的，所调用的系统调用命令是操作系统的一部分。

操作系统的发展与分类（理解）

1. 手工操作阶段（此阶段无操作系统）：
    - 用户在计算机上算题的所有工作都要人工干预，如程序的装入、运行、结果的输出等。
    - 随着计算机硬件的发展，人机矛盾（速度和资源利用）越来越大

2. 批处理阶段（操作系统开始出现）：

> 批处理系统是实现作业自动控制而**无须人工干预**的系统。

- **单道批处理**系统：
  - 系统对作业的处理是成批进行的，但内存中始终保持一道作业。
  - 目的是为了解决人机矛盾及 CPU 和 I/O 设备速率不匹配的矛盾
  - 主要特征：
    - 自动性：

      > 在顺利的情况下，磁带上的一批作业能自动地逐个运行，而**无须人工干预**。

    - 顺序性：

      > 磁带上的各道作业**顺序**地进入内存，各道作业的完成顺序与它们进入内存的顺序在正常情况下应完全相同。

    - 单道性：

      > 内存中仅有**一道**程序运行，即监督程序每次从磁带上只调入一道程序进入内存。

  - **多道批处理**系统：
    - 多道程序设计技术允许多个程序同时进入内存并允许它们在 CPU 中交替地运行，这些程序共享系统中的各种硬/软件资源。
      - 多道程序设计的特点：
        - 多道：

          > 计算机内存中同时存放多道相互独立的程序。

        - 宏观上并行：

          > 同时进入系统的多道程序都处于运行过程中，即它们先后开始各自的运行，但都未运行完毕。

        - 微观上串行：

          > 内存中的多道程序轮流占有 CPU，**交替执行**。

    > 优点：
    >
    > - **资源利用率高**，多道程序共享计算机资源，从而使各种资源得到充分利用。

- 系统**吞吐量大**，CPU 和其他资源保持「忙碌」状态。
- 缺点：
  - 用户响应的时间较长。
  - 不提供人机交互能力，用户既不能了解自己的程序的运行情况，又不能控制计算机。

3. **分时**操作系统：
   - 分时技术指把处理器的运行时间分成**很短的时间片**，按时间片轮流把 CPU 分配给各联机作业使用。
   - 若某个作业在分配给它的时间片内不能完成其计算，则该作业暂时停止运行，把处理器让给其他作业使用，等待下一轮再继续运行。
   - 由于计算机速度很快，作业运行轮转得也很快，因此给每个用户的感觉就像是自己**独占**一台计算机。
   - 多道批处理是实现作业自动控制而无须人工干预的系统，而分时系统是实现**人机交互**的系统。
   - 分时系统的主要特征如下：
     - **同时性（多路性）**：

       指允许多个终端用户同时使用一台计算机，即一台计算机与若干台终端相连接，终端上的这些用户可以同时或基本同时使用计算机。
     - **交互性**：

       用户能够方便地与系统进行人机对话，即用户通过终端采用人机对话的方式直接控制程序运行，与同程序进行交互。
     - **独立性**：

       系统中多个用户可以彼此独立地进行操作，**互不干扰**，单个用户感觉不到别人 也在使用这台计算机，好像只有自己单独使用这台计算机一样。
     - **及时性**：

       用户请求能在很短时间内获得响应。分时系统采用时间片轮转方式使一台计算机同时为多个终端服务，使用户能够对系统的及时响应感到满意。
4. 实时操作系统：
   - 为了能在某个时间限制内完成某些紧急任务而**不需要时间片排队**，诞生了实时操作系统。
   - 这里的时间限制可以分为两种情况：

> - 若某个动作必须**绝对地在规定的时刻**（或规定的时间范围）发生，则称为**硬实时**系统，如飞行器的飞行自动**控制**系统，这类系统必须提供绝对保证，让某个特定的动作在规定的时间内完成。
> - 若能够接受**偶尔**违反时间规定且不会引起任何永久性的损害，则称为**软实时**系统，如**飞机订票系统**、银行管理系统。
- 在实时操作系统的控制下，计算机系统接收到外部信号后及时进行处理，并在严格的时限内处理完接收的事件。

  > 实时系统必须能足够及时地处理某些紧急的外部事件，因此普遍用高优先级，并用「可抢占」来确保实时处理。
- 实时操作系统的主要特点是**及时性和可靠性**，允许以「浪费」一些资源为代价。

5. 网络操作系统和分布式计算机系统
6. 个人计算机操作系统

操作系统运行环境

内核态和用户态

- 计算机系统中，通常 CPU 执行两种不同性质的程序：
  - 操作系统**内核程序**
  - 用户自编程序（ 简称「**应用程序**」）
  - 前者是后者的管理者
- 因此「管理程序」（即内核程序）可以执行一些**特权指令**，而「被管理程序」（即用户自编程序）出于安全考虑不能执行这些指令。
- 特权指令是指计算机中不允许用户直接使用的指令，如 **I/O 指令，置中断指令，存取用于内存保护的寄存器，送程序状态字到程序状态字寄存器**等的指令。
- 在具体实现上，将 CPU 的状态划分为**用户态（目态）和核心态（又称管态、内核态）**。
- 操作系统的内核构成：
  - 与**硬件**关联较紧密的模块，如**时钟管理、中断处理、设备驱动**等处于最低层
  - 与**运行频率**较高的程序，如**进程管理、存储器管理和设备管理**。
  - 这两部分内容的指令操作工作在**核心态**。

  > 注意选择题中「执行」和「发生」的区别。

>
> <img src="./Operating System.assets/image-20211210220654769.png" alt="image-20211210220654769" style="zoom:100%;"/>
>
>
> <img src="./Operating System.assets/image-20211210224855348.png"/>

- 操作系统的内核的内容：
  - **时钟管理**：
    - 在计算机的各种部件中，时钟是最关键的设备。
    - 时钟的第一功能是计时。
    - 第二功能是通过时钟中断的管理，可以实现**进程的切换**。

    > 例如：
    >
    > - 在分时操作系统中采用时间片轮转调度，在实时系统中按截止时间控制运行。
    > - 在批处理系统中通过时钟管理来衡量一个作业的运行程度。
    >
    > >
    > > <img src="./Operating System.assets/image-20211210232945020.png"/>
  - **中断机制**：
    - 引入中断技术的初衷是**提高多道程序运行环境中 CPU 的利用率**，而且主要是针对外部设备的。
    - 后来逐步成为操作系统各项操作的基础。例如，键盘或鼠标信息的输入、进程的管理和调度、系统功能的调用、设备驱动、文件访问等。
    - 可以说，现代操作系统是靠中断驱动的软件。
    - 中断机制中，只有一小部分功能属于**内核**，它们**负责保护和恢复中断现场的信息**，转移控制权到相关的处理程序。
    - 这样可以减少中断的处理时间，提高系统的并行处理能力。
  - **原语**：
    - 原语定义：

      按层次结构设计的操作系统，底层是一些可被调用的公用小程序，它们各自完成一个规定的操作，特点如下：

- 处于操作系统的**最低层**，是最接近硬件的部分。
- 这些程序的运行具有**原子性**，其操作只能一气呵成（主要从系统安全性和便于管理考虑）。
- 这些程序的运行时间都较短，而且调用频繁。

通常把具有这些特点的**程序**称为**原语（Atomic Operation）**。

> 定义原语的直接方法是**关闭中断**，让其所有动作不可分割地完成后再**打开中断**。
>
> 系统中的**设备驱动、CPU 切换、进程通信**等功能中的部分操作都可定义为原语，使它们成为内核的组成部分。

- **系统控制的数据结构及处理**：

  系统中用来登记状态信息的数据结构很多，如**作业控制块、进程控制块（PCB）、设备控制块、各类链表、消息队列、缓冲区、空闲区登记表、内存分配表**等。为了实现有效的管理，系统需要一些基本的操作，常见的操作有以下 3 种：

  - **进程**管理：

    进程状态管理、进程调度和分派、创建与撤销进程控制块等。

  - **存储器**管理：

    存储器的空间分配和回收、内存信息保护程序、代码对换程序等。

  - **设备**管理：

    缓冲区管理、设备分配和回收等。

> 核心态指令实际上包括**系统调用类指令和一些针对时钟、中断和原语**的操作指令。

中断

- 中断（外中断）和异常（内中断）概念：

  - 系统不允许用户程序实现核心态的功能，而它们又必须使用这些功能，就得通过中断或异常实现。
  - 中断（Interruption）也称**外中断**，指来自 CPU **执行指令以外**的事件的发生，如**外设请求或人为干预**。
  - 异常（Exception）也称**内中断**、例外或陷入（trap），指源自 CPU **执行指令内部**的事件，如程序的非法操作码**地址越界、算术溢出、虚存系统的缺页及专门的陷入指令**等引起的事件。

> 对异常的处理一般要依赖于当前程序的运行现场，而且**异常不能被屏蔽**，一旦出现应立即处理。
>
> 所谓「缺页中断」属于**异常**，在考研题中，认为『「访存时缺页」属于中断』这句话是错误的。

- 中断处理过程：
  - **关中断**：
    - CPU 响应中断后，首先要保护程序的现场状态，在**保护现场**的过程中，CPU **不应响应更高级中断源的中断请求**。
    - 若现场保存不完整，在中断服务程序结束后，也就不能正确地恢复并继续执行现行程序。
  - **保存断点**：

    为保证中断服务程序执行完毕后能正确地返回到原来的程序，必须将原来的程序的断点（即**程序计数器 PC**）保存起来。
  - **中断服务程序寻址**：

    其实质是取出**中断服务程序的入口地址**送入程序计数器 PC。

    > 上面三步是在 CPU 进入中断周期后，由**硬件自动（中断隐指令）**完成的。
    >
    > 下面几步由中断服务程序完成。
  - **保存现场和屏蔽字**：

    进入中断服务程序后，首先要保存现场，现场信息一般是指**程序状态字寄存器 PSWR** 和某些通用寄存器的内容。
  - **开中断**：

    允许**更高级中断**请求得到响应。
  - 执行中断服务程序：

    这是中断请求的目的。
  - **关中断**：

    保证在**恢复现场和屏蔽字时不被中断**。
  - **恢复现场和屏蔽字**：

    将现场和屏蔽字恢复到原来的状态。
  - **开中断、中断返回**：

> 中断服务程序的**最后一条指令通常是一条中断返回指令**，使其返回到原程序的**断点**处，以便继续执行原程序。

> 易错点：
>
> 由**硬件**自动保存被中断程序的断点（即程序计数器 PC）。
>
>
> <img src="./Operating System.assets/image-20211210233357503.png"/>

系统调用

- 系统调用是指用户在程序中**调用**操作系统所提供的一些**子功能**，系统调用可视为特殊的公共子程序。
- 系统中的各种**共享资源**都由操作系统统一掌管，因此在用户程序中，凡是与资源有关的操作（如**存储分配、进行 I/O 传输及管理文件**等），都必须通过系统调用方式向操作系统**提出服务请求**，并由操作系统**代**为完成。
- 这些系统调用按功能大致可分为如下几类。
  - 设备管理：
    > 完成**设备的请求或释放，以及设备启动**等功能。
  - 文件管理：
    > 完成文件的**读、写、创建及删除**等功能。
  - 进程控制：
    > 完成进程的**创建、撤销、阻塞及唤醒**等功能。
  - 进程通信：
    > 完成**进程之间的消息传递或信号传递**等功能。
  - 内存管理：
    > 完成内存的**分配、回收以及获取作业占用内存区大小及始址**等功能。

> 操作系统的**运行环境**就可以理解为：
>
> - 用户通过操作系统运行**上层程序**（如系统提供的命令解释程序或用户自编程序），而这个上层程序的运行依赖于操作系统的底层管理程序提供服务支持。
> - 当需要管理程序服务时，系统则通过**硬件中断机制进入核心态**，运行管理程序

> - 当程序运行出现**异常**情况时，也会被动地需要管理程序的服务，这时就通过**异常处理**来进入核心态（**trap 指令在用户态执行**）。管理程序运行结束时，用户程序需要继续运行，此时通过相应的保存的程序现场，退出中断处理程序或异常处理程序，返回断点处继续执行。
>
> 一些由**用户态**转向**核心态**的例子：
>
> - 用户程序要求操作系统的服务，即**系统调用**。
> - 发生一次**中断**。
> - 用户程序中产生了一个错误状态。
> - 用户程序中企图执行一条特权指令（需要**访管中断**，访管指令不是特权指令）。
> - 从核心态转向用户态由一条指令实现，这条指令是**特权命令**，一般是中断返回指令。
>
> 执行系统调用的**过程**如下：
>
> 1. 正在运行的进程先传递系统调用参数
> 2. 在**用户态**执行**陷入指令**，将用户态转换为内核态，并将返回地址压入堆栈以备后用
> 3. CPU 执行相应的内核态服务程序
> 4. 最后返回用户态

大内核（宏内核）和微内核

- **大内核**系统将操作系统的**主要功能模块**都作为一个紧密联系的整体运行在**核心态**，从而为应用提供**高性能**的系统服务。
- 因为各管理模块之间**共享信息**，能有效利用相互之间的有效特性，所以具有无可比拟的性能优势。
- 为解决操作系统的内核代码难以维护的问题，提出了**微内核**的体系结构。它将内核中**最基本的功能（如进程管理等）保留在内核**，而将那些不需要在核心态执行的功能移到**用户态**执行，从而降低了内核的**设计复杂性**。
- 那些移出内核的操作系统代码根据分层的原则被划分成若干服务程序，它们的执行相互独立，**交互则都借助于微内核进行通信**。
- 微内核结构有效地分离了内核与服务、服务与服务，使得它们之间的接口更加清晰，维护的代价大大降低，各部分可以独立地优化和演进，从而保证了操作系统的**可靠性**。
- 微内核结构的最大问题是**性能问题**，因为需要频繁地在核心态和用户态之间进行切换，操作系统的执行开销偏大。

image-20210914200748917

进程管理

进程的概念和特征

> 来自《王道》

- 进程的概念：
  - 在多道程序环境下，允许多个**程序**并发执行，此时它们将**失去封闭性，并具有间断性及不可再现性**的特征。
  - 为此引入了**进程**（Process）的概念，以便更好地描述和控制程序的并发执行，实现操作系统的**并发性和共享性**（最基本的两个特性）。
  - 为了使参与并发执行的程序（含数据）能**独立地运行**，必须为之配置一个专门的数据结构，称为**进程控制块**（Process Control Block，PCB）。
  - 系统利用 PCB 来描述进程的**基本情况和运行状态**，进而控制和管理进程。
  - 相应地，由**程序段、相关数据段和 PCB **三部分构成了**进程映像（进程实体）**。
  - 所谓创建进程，**实质上是创建进程映像中的 PCB**；而撤销进程，实质上是撤销进程的 PCB。

    > 值得注意的是，**进程映像是静态的，进程则是动态的**。
    >
    > 注意：PCB 是**进程存在的唯一标志**。
- 进程的典型定义：
  - 进程是**程序的一次执行过程**。
  - 进程是**一个程序及其数据在处理机上顺序执行**时所发生的活动。
  - 进程是具有独立功能的**程序**在一个**数据**集合上运行的过程，它是系统进行**资源分配和调度的一个独立单位**。

    > 进程是**进程实体的运行过程**，是系统进行资源分配和调度的一个独立单位（基本单位）。
    >
    > 进程是由多道程序的并发执行而引出的，它和程序是两个截然不同的概念。
- 进程的特征（理解）：

  > 进程的基本特征是对比**单个程序的顺序执行**提出的，也是对进程管理提出的基本要求。

  - **动态性**：

> 进程是程序的一次执行，它有着创建、活动、暂停、终止等过程，具有一定的生命周期，是**动态地产生、变化和消亡**的。
>
> > 动态性是进程**最基本**的特征。

- **并发性**：

> 指**多个进程实体**同时存于内存中，能在一段时间内同时运行。
>
> > 并发性是进程的重要特征，同时也是操作系统的重要特征。引入进程的目的就是使程序能与其他进程的程序并发执行，以提高**资源利用率**。

- **独立性**：

> 指进程实体是一个能独立运行、独立获得资源和独立接受调度的**基本单位**。凡未建立 PCB 的程序，都不能作为一个独立的单位参与运行。

- **异步性**：

> 由于进程的相互**制约**，使得进程具有执行的**间断性**，即进程按**各自独立的、不可预知**的速度向前推进。
>
> > 异步性会导致执行结果的**不可再现性**，为此在操作系统中必须配置相应的进程同步机制。

- **结构性**：

> 每个进程都配置一个进程控制块 PCB 对其进行描述。
>
> > 从结构上看，**进程实体**（也可以说是进程）是由**程序段、数据段和 PCB** 三部分组成的。

- **进程和程序**的区别：

> - **本质区别**：
> - 进程是**动态**的；程序是**静态**的。
> - 进程是程序及其数据在计算机上的一次运行活动，是一个动态的概念。
> - 定义不同：
>
> 进程是程序处理数据的**过程**；程序是一组**指令的有序集合**。
>
> > 进程的运行实体是程序，离开程序的进程没有存在的意义。
>
> - 组成不同：

> 从静态角度看，进程是由程序、数据和进程控制块（PCB）三部分组成的；程序则是一组代码的集合。

- 生命周期不同：

> 进程是动态地创建和消亡的，具有**一定的生命周期**，是暂时存在的；程序则是一组代码的集合，是永久存在的，可长期保存。

- 对应关系不同：

> 一个进程可以执行一个或几个程序，一个程序也可构成多个进程。进程可创建进程，而程序不可能形成新的程序。

> 以下三点来自 CSAPP

- 逻辑控制流：

  - 即使在系统中通常有许多其他程序在运行，进程也可以向每个程序提供一种假象，好像它在**独占**地使用处理器。
  - 用调试器单步执行程序，会得到一系列的**程序计数器**（PC）的值，这些值唯一地对应于包含在程序的可执行目标文件中的指令，或是包含在运行时动态链接到程序的共享对象中的指令。这个 PC 值的序列叫做**逻辑控制流**，或者简称逻辑流。

    > 考虑一个运行着三个进程的系统，如图所示。处理器的一个物理控制流被分成了三个逻辑流，每个进程一个。每个竖直的条表示一个进程的逻辑流的一部分。在这个例子中，三个逻辑流的的关系是：
    >
    > - A 和 B 是**并发**执行的，A 和 C 也是并发执行的（如果两个逻辑流在**宏观时间上有重叠**，则称这两个进程是并发的）。
    > - B 和 C 是**顺序**执行的。
    >
    >
    >
    >
    >

- 私有地址空间：

  - **进程**也为每个**程序**提供一种假象，好像它**独占**地使用**系统地址空间**。
  - 进程为每个程序提供它自己的私有地址空间。一般而言，和这个空间中某个地址相关联的那个内存字节是不能被其他进程读或者写的，从这个意义上说，这个地址空间是私有的。
  - 尽管和每个私有地址空间相关联的内存的内容一般是不同的，但是每个这样的空间都有相同的通用结构。

- 上下文切换：

  - 操作系统内核使用一种称为**上下文切换**的较高层形式的**异常控制流**来实现多任务
  - 内核为每个进程维持一个上下文（context），上下文就是**内核重新启动一个被抢占的进程**所需的状态。
  - 它由一些**对象的值**组成，这些对象包括**通用目的寄存器、浮点寄存器、程序计数器、用户栈、状态寄存器、内核栈和各种内核数据结构**，比如描述地址空间的**页表**、包含有关当前进程信息的**进程表**，以及包含进程已打开**文件的信息的文件表**。
  - 在进程执行的某些时刻，内核可以决定**抢占当前进程**，并重启一个先前被抢占了的进程。这种决策就叫做**调度**，是由内核中称为**调度器**的代码处理的。
  - 在内核调度了一个新的进程运行后，它就抢占当前进程，并使用一种称为**上下文切换的机制**来将**控制**转移到新的进程：
    - **保存**当前进程的上下文。
    - **恢复**某个先前被抢占的进程被保存的上下文。
    - 将控制传递给这个新恢复的进程。
  - 当内核代表用户执行**系统调用**时，可能会发生上下文切换。如果系统调用因为等待某个事件发生而阻塞，那么内核可以让当前进程休眠，切换到另一个进程。

进程的状态与转换

> 来自《王道》

- 状态描述：

  进程在其生命周期内，由于系统中各进程之间的**相互制约关系**及系统的运行环境的变化，使得进程的状态也在不断地发生变化（**一个进程会经历若干不同状态**）。通常进程有以下 5 种状态，前 3 种是进程的基本状态。

  - 运行态：

    进程正在处理机上运行。在**单处理机环境**下，每个时刻最多只有**一个**进程处于运行态。

  - 就绪态：

    进程获得了**除处理机外的一切所需资源**，一旦得到处理机，便可立即运行。系统中处于就绪状态的进程可能有多个，通常将它们排成一个队列，称为就绪队列。

> 不可能所有进程都处于就绪态

- 阻塞态（等待态）：

  > 进程正在**等待某一事件**而暂停运行，如等待**某资源为可用**（不包括处理机）或等待**输入/输出完成**。即使处理机空闲，该进程也不能运行。

- 创建态：

  > 进程正在被创建，尚未转到就绪态。
  >
  > 创建进程通常需要多个步骤：
  >
  > 1. **申请一个空白的 PCB**，**并向 PCB 中填写一些控制和管理进程的信息**
  > 2. 由系统为该进程**分配运行时所必需的资源**
  > 3. 把该进程转入就绪态

- 结束态：

  > 进程正从系统中消失，可能是**进程正常结束或其他原因中断退出运行**。进程需要结束运行时，系统首先必须将该进程置为结束态，然后进一步处理资源释放和回收等工作。

- 状态转换：

  - 就绪态 $\to$ 运行态：

    > 处于就绪态的进程被调度后，获得**处理机资源（分派处理机时间片）**，于是进程由就绪态转换为运行态。

  - 运行态 $\to$ 就绪态：

    > - 处于运行态的进程在**时间片**用完后，不得不让出处理机，从而进程由运行态转换为就绪态。
    > - 在**可抢占**的操作系统中，当有更高优先级的进程就绪时，程序将正在执行的进程转换为就绪态，让更高优先级的进程执行。

  - 运行态 $\to$ 阻塞态：

    > 进程请求**某一资源（如外设）的使用和分配或等待某一事件的发生（如 I/O 操作的完成）**时，它就从运行态转换为阻塞态。

  - 阻塞态 $\to$ 就绪态：

    > 进程等待的事件到来时，如 I/O 操作结束或中断结束时，中断处理程序必须把相应进程的状态由阻塞态转换为就绪态。

进程控制

> 来自《王道》
>
> 在操作系统中，一般把进程控制用的程序段称为**原语**，原语的特点是执行期间**不允许中断**，它是一个不可分割的基本单位。

- 进程的创建：
  - 允许一个进程创建另一个进程。此时创建者称为**父进程**，被创建的进程称为子进程。
  - 子进程可以**继承父进程所拥有的资源**。当子进程被撤销时，应将其从父进程那里获得的资源**归还**给父进程。
  - 在撤销父进程时，必须同时撤销其所有的子进程。
  - 操作系统创建一个新进程的过程如下（创建原语）：
    - 为新进程分配一个唯一的**进程标识号**，并申请一个空白的 PCB（PCB 是有限的）。若 PCB 申请失败，则创建失败。
    - 为进程分配资源，为新进程的**程序和数据及用户栈分配必要的内存空间**（在 PCB 中体现）。

      > 若资源不足（如内存空间），则并不是创建失败，而是处于阻塞态，等待内存资源。
    - 初始化 PCB，主要包括初始化**标志信息**、**处理机状态与控制信息**，以及设置**进程的优先级**等。
    - 若进程就绪队列能够接纳新进程，则将新进程插入就绪队列，等待被调度运行。

    >
    > <img src="./Operating System.assets/image-20211213151050037.png"/>

- 进程的终止：
  - 引起进程终止的事件主要有：
    - 正常结束：

      > 表示进程的任务已完成并准备退出运行。
    - 异常结束：

      > 表示进程在运行时，发生了某种异常事件，使程序无法继续运行，如存储区越界、保护错、非法指令、特权指令错、运行超时、算术运算错、I/O 故障等。

- 外界干预：

  > 指进程应外界的请求而终止运行，如**操作员或操作系统干预**、**父进程请求**和**父进程终止**。

- 操作系统终止进程的过程如下（撤销原语）：
  - 根据被终止进程的标识符，**检索 PCB**，从中读出该进程的状态。
  - 若被终止进程处于执行状态，立即终止该进程的执行，将**处理机资源**分配给其他进程。
  - 若该进程还有子孙进程，则**应将其所有子孙进程终止**。
  - 将该进程所拥有的全部资源，或**归还**给其父进程，或归还给操作系统。
  - 将该 PCB 从所在队列（链表）中删除。

- 进程的阻塞和唤醒：
  - 正在执行的进程，由于期待的某些事件未发生，如**请求系统资源失败**、**等待某种操作的完成**、新数据尚未到达或无新工作可做等，由系统自动执行阻塞原语（Block），使自己由运行态变为**阻塞态**。

    > 进程的阻塞是进程自身的一种**主动行为**，也因此只有处于**运行态**的进程（获得 CPU），才可能将其转为阻塞态。

  - 阻塞原语的执行过程如下：
    - 找到将要被阻塞进程的标识号对应的 PCB。
    - 若该进程为运行态，则**保护其现场**，将其状态转为阻塞态，停止运行。
    - 把该 PCB 插入相应事件的等待队列，将处理机资源调度给其他就绪进程。

      > 当被阻塞进程所期待的事件出现时，如它所启动的 I/O 操作已完成或其所期待的数据已到达，由**有关进程**（比如，释放该 I/O 设备的进程，或提供数据的进程）调用**唤醒原语**（Wakeup），将等待该事件的进程唤醒。
      >
      > 唤醒原语的执行过程如下：
      >
      > - 在该事件的等待队列中找到相应进程的 PCB.
      > - 将其从等待队列中移出，并置其状态为就绪态。
      > - 把该 PCB 插入就绪队列，等待调度程序调度。
      > - 需要注意的是，Block 原语和 Wakeup 原语是一对作用刚好相反的原语，必须成对使用。

> > > >

- 子进程得到与父进程**用户级虚拟地址空间**相同的（但是**独立**的）一份**副本**，包括**代码和数据段、堆、共享库以及用户栈**。
- 子进程还获得与父进程任何打开文件描述符相同的副本，这就意味着当父进程调用 `fork()` 时，子进程可以读写父进程中打开的任何**文件**。
- 父进程和新创建的子进程之间**最大的区别**在于它们有**不同的 PID**。
- `fork()` 函数只被调用一次，却会返回两次。一次是在调用进程（父进程）中，一次是在新创建的子进程中：
  - 在**父进程**中，`fork()` 返回子进程的 PID。
  - 在**子进程**中，`fork()` 返回 0。因为子进程的 PID 总是**为非零**，返回值就提供一个明确的方法来**分辨**程序是在父进程还是在子进程中执行。

```
int fork(void);
```

- 子进程和父进程并发执行。
- 进程图：

  > 有向边 $a\to b$ 表示 a 在 b 前执行。

  对于代码：

```
int main(){
    Fork();
    Fork();
    printf ("hello\n");
    exit(0);
}
```

  其进程图为：

- 进程回收：
  - 当一个进程由于某种原因终止时，内核并不是立即把它从系统中清除，而是被保持在一种**已终止**的状态中，直到被它的**父进程回收**。
  - 当父进程回收已终止的子进程时，内核将子进程的**退出状态**传递给父进程，然后抛弃已终止的进程，从此时开始，该进程就不存在了。

- 一个终止了但还未被回收的进程称为**僵死进程**。
- 如果一个父进程终止了，内核会安排 **init 进程**成为它的孤儿进程的**养父**。
- init 进程的 PID 为 1，是在系统启动时由**内核**创建的，它不会终止，是**所有进程的祖先**。
- 如果父进程没有回收它的僵死子进程就终止了，那么内核会安排 init 进程去回收它们。
- 即使僵死子进程没有运行，它们仍然**消耗**系统的内存资源。
- 一个进程可以通过调用 `waitpid` 函数来等待它的子进程终止或者停止。

```
pid_t waitpid(pid_t pid, int *statusp, int options);
```

  - 默认情况下（当 `options = 0` 时），`waitpid` **挂起调用进程**的执行，直到它的等待集合中的**一个子进程**终止。
  - 如果等待集合中的**一个进程**在刚调用的时刻就已经终止了，那么 `waitpid` 就**立即返回**。
  - `waitpid` 返回导致 `waitpid` 返回的**已终止子进程的 PID**。已终止的子进程已经被回收，内核会从系统中删除掉它的所有痕迹。

    > 如果 `pid > 0`，那么等待集合就是一个单独的子进程，它的进程 ID 等于 pid。
    > 如果 `pid = -1`，那么等待集合就是由**父进程所有的子进程**组成的。

  - 调用 `wait(&status)` 等价于调用 `waitpid(-1, &status, 0)`。

  > 常用写法总结：
  >
  > - 循环回收并等待所有子进程结束：
  >
  > ```
  > while ((pid = waitpid(-1, &status, 0)) > 0)
  > ```
  >
  > - 循环回收当前所有已结束的子进程，不等待：
  >
  > ```
  > while (waitpid(-1, 0, WNOHANG) > 0)
  > ```
  >
  > > `WNOHANG`：
  > >
  > > - 如果等待集合中的任何子进程都还没有终止，那么就立即返回（返回值为 0）。

> > > >

- 进程创建时，操作系统为它新建一个 PCB，该结构之后常驻**内存**，任意时刻都可以存取，并在进程结束时删除。
- PCB 是进程实体的一部分，是**进程存在的唯一标志**。
- 进程执行时，系统通过其 PCB 了解进程的现行**状态信息**，以便对其进行控制和管理；进程结束时，系统收回其 PCB，该进程随之消亡。
- 操作系统通过 PCB 表来管理和控制进程：
  - 当操作系统欲**调度**某进程运行时，要从该进程的 PCB 中查出其**现行状态及优先级**。
  - 在调度到某进程后，要根据其 PCB 中所保存的处理机状态信息，设置该进程**恢复运行的现场**，并根据其 PCB 中的程序和数据的内存始址，找到其程序和数据。
  - 进程在**运行**过程中，当需要和与之合作的进程实现**同步、通信或访问文件**时，也需要访问 PCB。
  - 当进程由于某种原因而**暂停**运行时，又需将其断点的处理机环境保存在 PCB 中。
  - 可见，在进程的整个生命期中，系统总是通过 PCB 对进程进行控制的，亦即系统唯有通过进程的 PCB 才能感知到该进程的存在。

  > 下表是一个 PCB 的实例。

进程描述信息	进程控制和管理信息	资源分配清单	处理机相关信息
进程标识符（PID）	进程当前状态	代码段指针	通用寄存器值
用户标识符（UID）	进程优先级	数据段指针	地址寄存器值
	代码运行入口地址	堆栈段指针	控制寄存器值
	程序的外存地址	文件描述符	标志寄存器值
	进入内存时间	键盘	状态字
	处理机占用时机	鼠标	
	信号量使用		

- 程序段：

  程序段就是能被进程调度程序调度到 CPU 执行的程序代码段。注意，**程序可被多个进程共享**，即多个进程可以运行同一个程序。

- 数据段：

> 一个进程的数据段，可以是进程对应的程序加工处理的原始数据，也可以是程序执行时产生的中间或最终结果。

进程间通信

> 整理自《王道》
>
> 进程通信是指进程之间的信息交换。PV 操作是低级通信方式，高级通信方式是指以**较高的效率**传输大量数据的通信方式。

高级通信方法主要有以下三类:

- 共享存储：
  - 在通信的进程之间存在一块可直接访问的**共享空间**，通过对这片共享空间进行写/读操作实现进程之间的信息交换。
  - 在对共享空间进行写/读操作时，需要使用**同步互斥工具**（如 P 操作、V 操作），对共享空间的写/读进行控制。
  - 共享存储又分为两种：
    - 低级方式的共享是基于**数据结构**的共享。
    - 高级方式的共享则是基于**存储区**的共享。
  - 操作系统只负责为通信进程提供可共享使用的**存储空间和同步互斥工具**，而数据交换则由用户自己安排读/写指令完成。

    > 注意，用户进程空间一般都是独立的，进程运行期间一般不能访问其他进程的空间，只能通过共享空间间接访问，**不能通过全局变量共享**。
    >
    > 要想让两个用户进程共享空间，必须通过特殊的系统调用实现，而进程内的线程是自然共享进程空间的。
- 消息传递：
  - 在消息传递系统中，进程间的数据交换是以**格式化的消息（Message）为单位**的。
  - 若通信的进程之间不存在可直接访问的共享空间，则必须利用操作系统提供的消息传递方法实现进程通信。
  - 进程通过系统提供的**发送消息和接收消息两个原语**进行数据交换。
  - 有两种通信方式：
    - 直接通信方式：

      > 发送进程直接把消息发送给接收进程，并将它挂在接收进程的**消息缓冲队列**上，接收进程从消息缓冲队列中取得消息。

- 间接通信方式：

  > 发送进程把消息发送到某个**中间实体**，接收进程从中间实体取得消息。
  >
  > 这种中间实体一般称为信箱，这种通信方式又称信箱通信方式。该通信方式广泛应用于计算机网络，相应的通信系统称为电子邮件系统。

- 管道通信：
  - 所谓「**管道**」，是指用于连接一个读进程和一个写进程以实现它们之间的通信的一个**共享文件**，又名 pipe 文件。
  - 向管道（共享文件）提供输入的发送进程（即写进程），以字符流形式将大量的数据送入（写）管道。
  - 而接收管道输出的接收进程（即读进程）则从管道中接收（读）数据。
  - 为了协调双方的通信，管道机制必须提供以下三方面的协调能力：
    - **互斥**：

      > 一个进程正在对 pipe 进行读/写操作时，另一进程必须等待。

    - **同步**：
      - 当写（输入）进程把一定数量的数据写入 pipe 后，便去睡眠等待，直到读（输出）进程取走数据将其唤醒。
      - 当读进程读一空 pipe，也应睡眠等待，直至写进程将数据写入管道，才将其唤醒。
    - **确定对方的存在**：

      > 只有确定对方已存在时，才能进行管道通信，否则会造成因对方不存在而无限期等待。

  - 从本质上说，管道也是一种文件，但它又和一般的文件有所不同，管道可以克服使用文件进行通信的两个问题，具体表现如下：
    - 限制管道的大小：
      - 实际上，管道是一个**固定大小的缓冲区**，在 Linux 中，该缓冲区的大小为 $4\rm ~KB$，这使得它的大小不像文件那样不加检验地增长。
      - 使用单个固定缓冲区也会带来问题，比如在写管道时可能变满。
      - 这种情况发生时，随后对管道的 `write()` 调用将默认地被**阻塞**，等待某些数据被读取，以便腾出足够的空间供 `write()` 调用写。
    - 读进程也可能工作得比写进程快：

- 当所有当前进程数据已被读取时，管道变空。
- 这种情况发生时，一个随后的 `read()` 调用将默认地被阻塞，等待某些数据被写入，这解决了 `read()` 调用返回文件结束的问题。

> 从管道读数据是一次性操作，数据一旦被读取，它就从管道中被抛弃，释放空间以便写更多的数据。
>
> 管道只能采用**半双工通信**，即某一时刻只能单向传输，「双向数据传输」是错误的说法。要实现父子进程双方互动通信，需要定义两个管道。
>
>
> <img src="./Operating System.assets/image-20211211224246093.png"/>

异常控制流

> 整理自 PPT 和 CSAPP，以 CSAPP 的顺序描述

异常控制流概念

- 控制流定义：

  > 从开机到结束运行，**一个** CPU 所作的仅仅是读取并且执行（或者说是解释）**一连串的指令**，在某个时刻只会**运行一条指令**，这样一个指令的运行序列就是 CPU 的**控制流**。

- 异常控制流定义：

  > 现代系统通过使控制流发生突变来对这些情况做出反应，这些突变称为**异常控制流**（Exceptional Control Flow，ECF）。

- 异常定义：

  - 异常是异常控制流的一种**形式**，它一部分由**硬件**实现，一部分由**操作系统**实现。
  - 异常响应某些事件（即处理器状态更改），进而将控制权转移到 **OS 内核**。

- 异常处理：

  - 系统中可能的每种类型的异常都分配了一个**唯一的非负整数的异常号**：
    - 由**处理器的设计者**分配的号码：

      > 包括被零除、缺页、内存访问违例、断点以及算术运算溢出。

    - 由**操作系统内核**（操作系统常驻内存的部分）的**设计者**分配的号码：

包括**系统调用**和来自外部 I/O 设备的信号。

- 在任何情况下，当处理器检测到有事件发生时，它就会通过一张叫做**异常表**的跳转表，进行一个间接过程调用（异常），到一个专门设计用来处理这类事件的操作系统子程序（**异常处理程序**），当异常处理程序完成处理后，根据引起异常的事件的类型，会发生以下 3 种情况中的一种：
  - 处理程序将控制返回给当前指令 I_{curr}，即当事件发生时**正在执行**的指令。
  - 处理程序将控制返回给 I_{next}，即如果没有发生异常将会执行的**下一条**指令。
  - 处理程序**终止**被中断的程序。
- 异常类似于过程调用，但是有一些重要的不同之处：
  - 入栈内容不同：
    - 过程调用时，在跳转到处理程序之前，处理器将返回地址压入栈中。
    - 根据异常的类型，返回地址要么是当前指令，要么是下一条指令。
    - 处理器也把一些额外的**处理器状态**压到栈里，在处理程序返回时，重新开始执行被中断的程序会需要这些状态。
  - 入的栈不同：

    如果控制从用户程序转移到内核，所有这些项目都被压到**内核栈**中，而不是压到用户栈中。
  - 权限不同：

    异常处理程序运行在内核模式下，这意味着它们对所有的系统资源都有完全的访问权限。
- 异常类别：
  - 属性概括：

类别	原因	异步/同步	返回行为
中断（Interrupt）	来自 I/O 设备的信号	异步	总是返回到下一条指令
陷阱（Trap）	有意的异常	同步	总是返回到下一条指令
故障（Fault）	潜在的可恢复的错误	同步	返回到当前指令或终止
终止（Abort）	不可恢复的错误	同步	不会返回

  - 中断：
    - 中断是**异步**发生的，是来自处理器外部的 I/O 设备的信号的结果。

- 硬件中断**不是由任何一条专门的指令**造成的，从这个意义上来说它是异步的。
- 硬件中断的异常处理程序常常称为**中断处理程序**。
- 通过设置 CPU 的**中断引脚**来触发。

> 其他三类同步异常也叫故障指令。

- 陷阱：
  - 陷阱是**有意的**异常，是执行一条指令的结果。
  - 就像中断处理程序一样，陷阱处理程序将控制返回到**下一条指令**。
  - 陷阱最重要的用途是在**用户程序和内核**之间提供一个像过程一样的**接口**，叫做**系统调用**。
  - 系统调用运行在内核模式中，内核模式允许系统调用执行特权指令，并访问定义在内核中的栈。
- 故障：
  - 故障由错误情况引起，它**可能**能够被故障处理程序修正。
  - 当故障发生时，处理器将控制转移给**故障处理程序**。
  - 如果处理程序能够修正这个错误情况，它就将控制返回到引起故障的指令，从而**重新执行**它。
  - 否则，处理程序返回到内核中的 abort 例程，**终止**引起故障的应用程序。

> **缺页**和**无效的内存引用**（越界）会引发故障。

- 终止：

  终止是不可恢复的致命错误造成的结果，通常是一些**硬件错误**。

信号

- 信号定义：
  - 信号是**内核**发出的一个消息，用来通知进程，系统中发生了某种类型的事件。
  - 信号从**内核发往进程**（有时信号的发送是由另一个**进程发起**的）。
  - 每一种信号都用一个整数 ID 来表示。
  - 通常情况下只有信号 ID 会被发送给进程。
  - 使用信号的两个主要目的为：
    - 通知进程某种特殊的事件发生了（比如一些低层的**硬件异常**）。

    - 迫使进程**执行信号处理程序**。

- 进程组：
  - 每个进程都**只属于一个进程组**，进程组是由一个正整数进程组 ID 来标识的。`getpgrp` 函数返回当前进程的进程组 ID：

    ```
    pid_t getpgrp (void);
    pid_t getpgid(pid_t pid); /* 返回 ID 为 pid 的进程的组 ID */
    ```

  - 默认子进程和它的**父进程**同属于一个进程组，可以修改指定进程的进程组：

    ```
    /* 将进程 pid 的进程组改为 pgid,若成功则为 0,若错误则为 -1 */
    int setpgid (pid_t pid, pid_t pgid)
    /*
    * 如果 pid 是 0,那么就使用当前调用进程的 PID
    * 如果 pgid 是 0,那么就用 pid 指定的进程的 PID 作为进程组 ID
    * 例如:
    *   如果当前调用进程 PID 为 15213,那么 setpgid(0, 0) 会创建一个新的进程组,
    * 其进程组的 ID 是 15213,并且把进程 15213 加入到这个新的进程组中。
    */
    ```

  - `kill` 函数：

    ```
    int kill(pid_t pid, int sig)
    /*
    * pid > 0: 发送信号 sig 给进程 pid
    * pid = 0: 发送信号 sig 给调用进程的进程组内的所有进程包括其自身
    * pid < 0: 发送信号 sig 给进程组 |pid| (pid的绝对值) 内的所有进程
    */
    ```

  - 用 `/bin/kill` 程序发送信号：
    - `/bin/kill` 程序可以向另外的进程发送任意的信号。
    - 命令 `linux>/bin/kill -9 15213` 发送信号 `9(SIGKILL)` 给进程 `15213`。
    - 一个为负的 PID 会导致信号被发送到进程组 PID 中的每个进程。

> 传送一个信号到目的进程是由发送信号和接收信号完成。

- **发送信号**：
  - 内核通过更新目的**进程上下文中的某个状态**，发送一个信号给目的进程。
  - 发送信号可以有如下两种原因：
    - 内核检测到一个**系统事件**，比如除零错误或者子进程终止。
    - 一个进程调用了 `kill` 函数，显式地要求内核发送一个信号给目的进程。
  - 一个进程可以发送信号给它**自己**。
- **接收信号**：
  - 当目的进程被内核**强迫**以某种方式对信号的发送**做出反应**时，它就接收了信号。
  - 当内核把**进程 p** 从**内核模式切换到用户模式**时（例如，从系统调用返回或是完成了一次上下文切换），它会检查进程 p 的**未被阻塞的待处理信号**的集合（pending & ~blocked）：
    - 如果这个集合**为空**（通常情况下），那么内核将控制传递到进程 p 的逻辑控制流中的**下一条指令** $I_{{next }}$。
    - 如果集合是**非空**的，那么内核选择集合中的某个信号 k（通常是最小的 k），并且强制进程 p 接收信号 k，收到这个信号会触发进程采取某种行为。
    - 完成这一行为后将该位清零，然后重复这一过程，直到集合为空，把控制传递回 p 的逻辑控制流中的**下一条**指令 $I_{{next }}$。
    - 每个信号类型都有一个预定义的默认行为，是下面中的一种:
      - 进程终止。
      - 进程终止并**转储内存**。
      - 进程**停止（挂起）**直到被 SIGCONT 信号重启。
      - 进程**忽略**该信号。
  - 设置信号处理程序：

    ```
    sighandler_t signal(int signum, sighandler_t handler);
    ```

    `signal` 函数可以通过下列三种方法之一来改变和信号 `signum` 相关联的行为：
    - 如果 `handler` 是 `SIG-IGN`，那么忽略类型为 `signum` 的信号。
    - 如果 `handler` 是 `SIG-DFL`，那么类型为 `signum` 的信号行为恢复为默认行为。

    - 否则，`handler` 就是用户定义的函数的地址，这个函数被称为信号处理程序，只要进程接收到一个类型为 `signum` 的信号，就会调用这个程序。
    - 当一个进程捕获了一个类型为 k 的信号时，会调用为信号 k 设置的处理程序。
- 待处理信号：
    - 一个发出而没有被接收的信号叫做**待处理信号**。
    - 在任何时刻，**一种类型至多只会有一个**待处理信号。
    - 如果一个进程有一个**类型为 k 的待处理信号**，那么任何接下来发送到这个进程的类型为 k 的信号**都不会排队等待**，而是被简单地丢弃。
- 一个进程可以**有选择性地阻塞**接收某种信号。当一种信号被阻塞时，它仍可以被传送，但是它不会被接收，直到进程取消对这种信号的阻塞。
    - 一个待处理信号最多只能被接收一次。
    - 内核为每个进程在 pending **挂起位向量**中维护着**待处理信号**的集合，而在 blocked **阻塞位向量**中维护着被**阻塞**的信号集合。
    - 只要**传送**了一个类型为 k 的信号，内核就会**设置** pending 中的第 k 位。
    - 只要**接收**了一个类型为 k 的信号，内核就会**清除** pending 中的第 k 位。
- 阻塞信号：
    - **隐式**阻塞机制：

      内核默认阻塞任何当前处理程序**正在处理信号类型**的待处理的信号。
    - **显式**阻塞机制：

      应用程序可以使用 `sigprocmask` 函数和它的辅助函数，明确地阻塞和解除阻塞选定的信号。

```
/* sigset_t 类型其实是和 pending 和 blocked 相同的位向量，每一位表示一个信号 */
/* 如果成功则为 0，若出错则为 -1 */
int sigemptyset(sigset_t *set);           /* 初始化 set 为空集合 */
int sigfillset(sigset_t *set);            /* 把每个信号都添加到 set 中 */
int sigaddset(sigset_t *set, int signum); /* 把指定的信号 signum 添加到 e
*/
int sigdelset(sigset_t *set, int signum); /* 从 set 中删除指定的信号
signum */

/*
```

```
*  如果 how = SIG_BLOCK,则表示将 set 中的信号全部添加到 blocked 中,即阻塞 set 中的全部信号
*  将 blocked 位向量之前的值保存在 oldset 中,通过 sigprocmask(SIG_SETMASK, &oldset, NULL); 还原
*/
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);


/* 若 signum 是 set 的成员则为 1, 如果不是则为 0, 若出错则为 -1 */
int sigismember(const sigset_t *set, int signum);
```

编写信号处理要注意的问题

> 存在的问题：
>
> 信号处理程序与主程序**并发**运行，共享同样的全局变量，因此可能与主程序和其他处理程序互相干扰。

- **安全的信号处理**：
  - $G0$：

    处理程序尽可能地**简单**。

    > 例如：
    >
    > - 处理程序可能只是简单地设置全局标志并立即返回
    > - 所有与**接收信号**相关的处理都由**主程序**执行，**处理程序**只周期性地**检查**（并重置）这个标志。
  - $G1$：

    在处理程序中只调用**异步信号安全**的函数：

    - 是**可重入**的，参见 后文。
    - **不能被信号处理程序中断**。

    > 信号处理程序中产生**输出**唯一安全的方法是使用 `write` 函数。特别地，调用 `printf` 或 `sprintf` 是**不安全**的。
    >
    > 一些安全的函数，称为 SIO（安全的 I/O）包，可以用来在信号处理程序中打印简单的消息。
  - $G2$：

保存和恢复 `errno`。

- 许多 Linux 异步信号安全的函数都会在**出错返回时设置** `errno`。
- 在处理程序中调用这样的函数可能会干扰**主程序中其他依赖**于 `errno` 的部分。
- 解决方法是在进入处理程序时把 `errno` 保存在一个**局部变量**中，在处理程序返回前恢复它。
- 注意，只有在处理程序要返回时才有此必要。如果处理程序调用 `exit` 终止该进程，那么就不需要这样做了。
- 每个线程都有自己的局部 `errno`，是多线程安全的。

- $G3$：

  阻塞所有的信号，保护对**共享全局数据结构**的访问。

- $G4$：

  用 `volatile` 声明**全局变量**。

  > 用 `volatile` 类型限定符来定义一个变量，编译器就不会缓存它。用处在于：
  >
  > - 对于一个主程序和信号处理程序共享的变量 x，如果主程序没有对该变量进行写操作，它的值对于编译器来说就像一个定值
  > - 此时，一个优化编译器会认为使用缓存在寄存器中的副本来满足对 x 的每次使用是很安全的，但是实际上信号处理程序有可能改变 x。
  > - 如果这样，主程序可能永远都无法看到处理程序更新过的值。使用 `volatile` 类型就可以强制让编译器不对 x 进行缓存优化。
  >
  > 例如：
  >
  > `volatile int x;`
  >
  > 强迫编译器每次在代码中引用 `x` 时，都要从内存中读取 `x` 的值。
  >
  > 一般来说，和其他所有共享数据结构一样，应该暂时阻塞信号，保护每次对全局变量的访问。

- $G5$：

  用 `sig_atomict_t` 声明标志。

  > 在常见的处理程序设计中，处理程序会写**全局标志**来记录收到了信号。
  >
  > 主程序周期性地**读**这个标志，响应信号，再**清除**（写）该标志。

> > > 对于通过这种方式来共享的标志，C 提供一种整形数据类型 `sig_atomic_t`，对它的读和写保证会是原子的（不可中 断）。
> > > 对多条指令实现的 `flag++` 和 `flag = flag + 10` 不适用。

- 正确的信号处理：
  - 信号的一个与直觉不符的方面是未处理的信号是**不排队**的。
  - 因为 pending 位向量中每种类型的信号只对应有一位，所以每种类型最多只能有一个未处理的信号。
  - 所以要注意，如果存在一个未处理地信号就表明**至少有一个**信号到达了。
- 可移植的信号处理

CPU 调度

> 整理自 PPT，《王道》和《操作系统概念》

CPU 调度概念

- 在多道程序系统中，进程的数量往往多于处理机的个数。
- CPU 调度是对 CPU 并行分配，即从就绪队列中按照一定的算法（公平、高效）选择一个进程并将 CPU 分配给它运行，以实现进程并发地执行。
- CPU 调度是多道程序操作系统的基础，是操作系统设计的核心问题进程调度。
- 遵循的原则：
  > **资源高效，公平合理**（两者可能存在冲突）。
- CPU - I/O 执行周期：
  - 程序代码可以分为计算类代码和 I/O 类代码。
  - 进程执行过程由 CPU 执行和 I/O 等待周期组成 CPU 区间和 I/O 区间。
  - CPU 约束型（密集）程序以计算为主，CPU 区间较多，还会有少量长的 CPU 区间。
  - I/O 约束型程序以 I/O 为主，但配合 I/O 处理有大量短的 CPU 区间。
  - 进程通常具有**大量短 CPU 区间和少量长 CPU 区间**。
- 进程调度方式：
  > > 整理自《操作系统概念》
  > > 进程调度的时机：

> 1. 当一个进程从**运行**状态切换到**阻塞**状态时（例如，I/O 请求，或 `wait` 调用以便等待一个子进程的终止）
> 2. 当一个进程**终止**时
> 3. 当一个进程从**运行**状态切换到**就绪**状态时（例如，当出现时钟中断时，笔者感觉这个更像调度后的结果😒）
> 4. 当一个进程从**阻塞**状态切换到**就绪**状态时（例如，I/O 完成）

- 非剥夺调度方式，又称**非抢占方式**：

  > - 仅发生在情况 $1,2$。即一旦某个进程分配到 CPU，该进程就会一直使用 CPU，直到它**终止或切换到阻塞**状态。
  > - 当一个进程正在 CPU 上执行时，即使有某个更为重要的进程进入**就绪队列**，仍然让正在执行的进程**继续执行**，直到该进程**终止或切换到阻塞**状态时，才把 CPU 分配给更为重要的进程。
  >
  > > 优点：
  > >
  > > 实现简单、系统开销小，适用于大多数的**批处理系统**，但它不能用于**分时系统和大多数的实时系统**。

- 剥夺调度方式，又称**抢占方式**：

  > - 可以发生在情况 $1,2,3,4$。
  > - 当一个进程正在 CPU 上执行时，若有更重要的进程需要进入就绪队列，则**立即暂停**正在执行的进程，将 CPU 分配给这个更为重要的进程。
  >
  > > 采用剥夺式的调度，对**提高系统吞吐率和响应效率**都有明显的好处。
  > >
  > > 但「剥夺」不是一种任意性行为，必须遵循一定的原则，主要有**优先权、短进程优先和时间片原则**等。

调度的基本准则

- CPU 利用率：

  > CPU 是计算机系统中最重要和昂贵的资源之一，所以应尽可能使 CPU 保持「忙」状态，使这一资源利用率最高。

- 系统吞吐量：

  > 表示**单位时间内 CPU 完成作业的数量**。
  >
  > > - **长作业**需要消耗的处理机时间较**长**，因此会**降低**系统的吞吐量。

> - **短作业**需要消耗的处理机时间较**短**，因此能**提高**系统的吞吐量。
> - **调度算法和方式**的不同，也会对系统的吞吐量产生较大的影响。
> - 系统吞吐量大 $\Leftrightarrow$ CPU 使用率高 $+$ 上下文切换代价小

- 周转时间：

> 周转时间是指**从作业提交到作业完成**所经历的时间。
>
> - 作业的周转时间可用公式表示如下：
>
> > 周转时间 $=$ 作业完成时间 $-$ 作业提交时间
>
> - **平均周转时间**是指多个作业周转时间的平均值：
>
> > 平均周转时间 $=$（作业 1 的周转时间 $+\cdots+$ 作业 n 的周转时间）/ n

- 等待时间：

> 等待时间指**进程处于等处理机状态的时间之和**，等待时间越长，用户满意度越低。
>
> - CPU 调度算法实际上并**不影响作业执行或 I/O 操作**的时间，只影响作业在就绪队列中**等待**所花的时间。
> - 因此衡量一个调度算法的优劣，常常只需考察等待时间。

- 响应时间：

> 响应时间指从**用户提交请求到系统首次产生响应**所用的时间。
>
> > 在交互式系统中，周转时间不可能是最好的评价准则，一般采用响应时间作为衡量调度算法的重要准则之一。
> >
> > 从用户角度来看，调度策略应尽量降低响应时间，使响应时间处在用户能接受的范围之内。

> 公平性：
>
> 合理分配 CPU，使它尽可能的「忙」。
>
> 矛盾：
>
> - 响应时间短和公平性之间的矛盾：
>
> > **响应时间短** $\Rightarrow$ **前台**任务的**优先级高 **$\Rightarrow$ **后台任务**得不到 CPU
>
> - 吞吐量和响应时间之间的矛盾：

> 吞吐量大 $\Rightarrow$ 上下文切换代价小（尽量少切换） $\Rightarrow$ 时间片大 $\Rightarrow$ 响应时间长

调度算法

- 先到先服务（First - Come First - Served，FCFS）调度：

> 调度的顺序就是任务到达就绪队列的顺序。
>
> > 实现简单，但缺点为：
> >
> > - 平均等待时间往往很长，是非抢占的。
> > - 看似公平，实则对**长作业**比较有利，但对短作业不利（相对 SJF 调度和高响应比）
> >
> > > 一个先到的长作业会让短作业等待时间过长。
> >
> > 有利于 **CPU 繁忙型**作业，而不利于 I/O 繁忙型作业

- 最短作业优先（Shortest - Job - First，SJF）调度：

> - 非抢占式：
>
> > 从就绪队列中选择一个或若干**估计运行时间最短**的作业，将它们调入内存运行。
>
> - **抢占** SJF 调度，又称**最短剩余时间优先**（Shortest - Remaining - Time - First，SRTF）调度：
>
> > 会抢占当前运行进程，每次**新进程到来或者当前进程结束**时判断是否抢占当前进程。
>
> > 在执行时间确定的情况下，平均等待时间、平均周转时间最少。
> >
> > 缺点：
> >
> > - 对**长作业**不利，导致「饥饿」
> > - 完全未考虑作业的**紧迫程度**
> > - 运行时间为**估计值**，并不能反映真实运行时间，实际上可能做不到最短作业优先
>
> - 使用预测算法改进：
>
> > - 可以认为下一个 CPU 执行的长度与以前的相似。
> > - 下次 CPU 执行通常预测为以前 CPU 执行的测量长度的**指数平均**。

- 设 t_{n} 为第 n 个 CPU 执行长度，设 τ_{n+1} 为下次 CPU 执行预测值。因此，对于 $\alpha, 0 \leqslant \alpha \leqslant 1$，定义

$$\tau_{n+1} = \alpha t_n + (1-\alpha)\tau_n$$

  t_{n} 包括**最近信息**，而 τ_{n} 存储了**过去历史**，参数 α 控制最近和过去历史在预测中的**权重**。
- 如果 $\alpha=0$，那么 $\tau_{n+1}=\tau_{n}$，最近信息没有影响。
- 如果 $\alpha=1$，那么 $\tau_{n+1}=$ t_{n}，只有最近 CPU 执行才重要。
- 通常取 $\alpha=$ $1 / 2$，认为最近信息和过去历史同样重要。初始值 τ_{0} 可作为常量或系统的总体平均值。

- 优先级调度（SJF 的一般化）：
  - 每个进程都有一个**优先级**与其关联，而具有最高优先级的进程会分配到 CPU。
  - 进程优先级的设置可以参照以下原则：
    - **系统进程** $>$ 用户进程。系统进程作为系统的管理者，应拥有更高的优先级。
    - **交互型进程** $>$ 非交互型进程（前台进程 $>$ 后台进程）。在**前台运行**的正在和用户交互的进程应**更快得到响应**，因此需要更高的优先级。
    - **I/O 型**进程 $>$ 计算型进程。I/O 型进程是指会频繁使用 I/O 设备的进程，而计算型进程是那些频繁使用 CPU 的进程。I/O 设备（如打印机）的处理速度要比 CPU 慢得多，因此若将 I/O 型进程的优先级越高，就可以让 I/O 设备**越早开始工作**，进而提升系统的**整体效率**。
  - 会导致某些低优先级进程处于**饥饿**状态，不能运行，可以用**老化**技术逐渐增加在系统中等待很长时间的进程的优先级。
- （时间片）轮询（轮转）调度（Round - Robin，RR）：
  - 时间片轮转调度算法主要适用于**分时系统**。

    > 分时系统 中，多个用户分享使用同一台计算机。

  - 系统将所有就绪进程按**到达时间的先后**次序排成一个队列，进程调度程序总是选择**就绪队列中的第一个进程**执行，即**先来先服务**的原则，但**仅能运行一个时间片**。
  - 在使用完一个时间片后，进程即使未完成运行，也必须释放出 CPU 给下一个**就绪**的

进程，然后返回到就绪队列的**末尾**重新排队，等候再次运行。

- 时间片的大小对系统性能的影响很大：
  - 若时间片足够大，以至于所有进程都能**在一个时间片内执行完毕**，则时间片轮转调度算法就退化为**先来先服务调度算法**。
  - 若时间片很小，则 CPU 将在进程间过于频繁地切换，使处理机的**开销增大**，而真正用于运行用户进程的时间将减少。
- 时间片的长短通常由以下因素确定：
  - 系统的**响应时间**
  - **就绪队列**中的**进程数目**
  - 系统的处理能力

- 多级（反馈）队列调度：
  - 设置多个**就绪队列**，并为各个队列赋予不同的优先级，第 1 级队列的优先级最高，第 2 级队列次之，其余队列的优先级逐次降低。
  - 优先级越低的队列运行时间片越大，让长作业在低优先级的队列中得到更长时间的执行。
  - 将任务按照优先级分为多个队列，每次从优先级最高的队列选择任务。
  - 允许进程在队列之间移动，根据不同 CPU 执行的特点来区分进程。如果进程使用过多的 CPU 时间，那么它会被移到更低的优先级队列。这种方案将 **I/O 密集型和交互进程放在更高优先级队列**上。

    > 《操作系统概念》提及：
    >
    > 「在较低优先级队列中等待过长的进程会被移到更高优先级队列（老化）」
    >
    > 具体实现并没有给出例子，其他书也没有出现类似理解

  -
    - 最先执行队列 0 的任务，队列 0 为**空**时才执行队列 1，依此类推。
    - 到达高优先级队列的任务，会**抢占**低优先级队列的任务。
    - 一个新进程进入内存后，首先将它放入队列 0 的末尾，按 FCFS 原则排队等待调度。当轮到该进程执行时，如它能在该时间片内完成，便可准备撤离系统；否则将该进程转入队列 1 的末尾。
  - 综合了前几种调度算法的优点：
    - **短作业优先**。

> > > - **周转时间较短**。
> > > - 经过前面几个队列得到部分执行，长批处理作业不会长期得不到处理。

- 彩票调度：
  - 保证每个任务都获得**一定比例的 CPU 时间**。
  - 彩票数表示了任务应该接受到的资源份额
  - 彩票数百分比表示了其所占有的系统资源份额，随着运行时间的增加，任务得到的 CPU 时间比例会越接近该百分比。

CPU 优化

> 整理自 PPT 和 CSAPP

CPU 优化要求和影响因素

- 编写高效程序要求：
  - 选择适当的算法和数据结构。
  - 让编译器能够有效优化（编写**编译器友好**的代码）：
    - 编写程序方式中看上去只是一点小小的变动，都会引起编译器优化方式很大的变化。
    - 在程序的**可读性**模块性与**运行速度**之间做出权衡。
- 编译器会受到「妨碍优化的因素」的影响：
  - 潜在的**内存别名**使用：

    不同的指针指向同一个位置：

```
void fun1(long *xp, long *yp) {
    *xp += *yp;
    *xp += *yp
}
void fun2(long *xp, long *yp) {
    *xp += 2* *yp;
}
/*
* fun2 读写次数比 fun1 少,看起来功能一样,但当 yp 和 xp 指向同一内存单元时,结果会不同
* 我们希望编译器可以将 fun1 优化成 fun2,但是编译器要进行安全的优化,所以必须假设存在
```

```
上面这种特殊情况,故妨碍了优化。
*/
```

- 潜在的**函数副作用**:

  可能修改了全局程序状态

```
long counter=0;      //全局变量
long f() {
    return counter++;
}
long fun1() {
    return f()+f()+f()+f();
}
long fun2() {
    return 4*f();    // 与 fun1 在该情况下不等效,妨碍编译器优化
}
// 内联函数替换 fun1,减少函数调用的开销
long fun1inline() {
    long t=counter++;
    t+=counter++;
    t+=counter++;
    t+=counter++;
    return t;
}
```

- 编译器优化选项:
  - `-Og` 使用一组基本的优化。
  - `-O1` 有限的保守的优化。
  - `-O2` 适中的优化，对大多数项目来说，是可接受的标准。
  - `-O3` 更激进的优化，提高性能，增加规模，调试工具更难调试。
- 每元素的周期（Cycles Per Element，CPE）：

  用来评价循环性能，并不是实际上一个元素占用的时钟周期数，是个平均指标。

```
void psum1(float a[], float p[], long n) {
    long i;
    p[0] = a[0];
```

```
    for (i = 1; i < n; i++)
        p[i] = p[i-l] + a[i];
}

void psum2(float a[], float p[], long n) {
    long i;
    p[0] = a[0];
    for (i = 1; i < n-1; i+=2) {
        p[i] = p[i-1] + a[i];
        p[i+l] = p[i] + a[i+l];
    }
    /* For even n, finish remaining element */
    if (i < n)
        p[i] = p[i-l] + a[i];
}
```

- `psum2` 使用了循环展开的技术，CPE 比 `psum1` 要低，即每个元素需要的时钟周期数要比 `psum1` 要少，效率更高。
- $T($ 运行时间 $) = CPE\times n ~+$ 基本的周期开销

利用指令级并行进行优化

> 其实没太懂这些数是怎么得到的，感觉不怎么能考，听说有的班直接没讲😢

- 现代计算机的设计：
  - 硬件可以**并行执行**多个指令（每个时钟周期执行多个操作，乱序）。
  - 性能受**数据依赖**的限制。
  - 简单的转换可以带来显著的性能改进：
    - 编译器通常无法进行这些转换。
    - 浮点运算缺乏结合性和可分配性。
- 超标量定义：

  超标量指可以在每个时钟周期执行多个操作，而且是乱序的（指令执行的顺序与它们在机器级程序中的顺序可能不一样）。
- 名词定义：

  > Haswell 架构的 CPU 各模块数量：
  >
  > - 2 个加载功能模块

> - 1 个存储
> - 4 个整数运算
> - 2 个浮点乘法运算
> - 1 个浮点加法
> - 1 个浮点除法

- 延迟：

  > **完成运算**所需要的时钟周期总数。

- 发射时间：

  > **连续同类型运算**之间间隔**最小周期数**。发射为 1 意味着同类运算间不用等待。

- 容量：

  > 能够**执行该运算的功能单元数量**，即同时能发射多少这样运算。

- 延迟界限：

  > 一种运算的延迟为 n，意味着需要 n 个时钟周期执行这种运算，当**顺序执行**时，无论怎样优化，其 CPE 的下限不会低于延迟界限。

- 吞吐量界限：

  > CPE 的**最小界限**，假设执行某种运算的功能单元（**容量**）有 k 个（还要受限于**地址加载单元数量**），其**发射**为 n，则**吞吐量界限**为 $\begin{aligned}\frac{n}{k}\end{aligned}$。
>
> > 这里忽略延迟，其实是当延迟为 1 算，这样缩放可以让这个值成为真正的极限，无论怎么优化，CPE 都不可能比吞吐量界限小。
>
> image-20211012222645387

- 运算重组：

```
void fun1(){
    for (i = 0; i < limit; i++)
        x = x OP d[i];
}

// 2*1
void fun2(){
```

```
    for (i = 0; i < limit; i+=2)
        x = (x OP d[i]) OP d[i+1];
}

// 2*1a
void fun3(){
    for (i = 0; i < limit; i+=2)
        x = x OP (d[i] OP d[i+1]); // 消除了一个数据依赖,可以提前算 d[i] OP d[i+1],实现并行
}

// 2*2
void fun4(){
    for (i = 0; i < limit; i+=2)
        x0 = x0 OP d[i],
        x1 = x1 OP d[i+1];
}
```

方法	整数	整数	浮点数	浮点数	优化原因
操作	$+$	$\times$	$+$	$\times$	
`fun1`	1.27	3.01	3.01	5.01	
`fun2(2*1)`	1.01	3.01	3.01	5.01	整数加法减少了循环开销（一次循环也需要一次整数加法），但是和其他运算一样不能突破**延迟界限**
`fun3(2*1a)`	1.01	1.51	1.51	2.51	这里整数加法并没有得到优化，猜想编译器只用到了一个整数加功能模块，所以对于延迟为 1 的整数加没有提升
`fun4(2*2)`	0.81	1.51	1.51	2.51	用到了 2 个整数加功能模块，其他操作因为循环开销，延迟和数据依赖的问题，在单个功能模块时已达到性能极限
延迟界限	1.00	3.00	3.00	5.00	
吞吐量界限	0.50	1.00	1.00	0.50	

- 达到某操作的吞吐量界限所要求的**循环展开因**子：

> 达到时，要保持执行该操作的所有功能单元的**流水线都是满**的（即发射为 1），对延迟为 L，容量为 C 的操作而言，循环展开因子满足 $k\ge CL$。

线程

线程概念

> 整理自 PPT，CSAPP 和《王道》

线程与进程

> 引入进程的目的是更好地使多道程序并发执行，提高资源利用率和系统吞吐量。

- 引入线程的目的是**减小程序在并发执行时所付出的时空开销**，提高操作系统的并发性能。
- 线程可理解为「轻量级进程」，它是一个基本的 CPU 执行单元，也是程序执行流的**最小单元**，由线程 ID、程序计数器、寄存器集合和堆栈组成。

  > 线程包含 CPU 现场，可以独立执行程序

- 线程是进程中的一个实体，是被**系统独立调度和分派的基本单位**，是进程内一个**相对独立的可调度的执行单元**。
- 线程自己不拥有系统资源，只拥有一点儿在运行中必不可少的资源，但它可与同属一个进程的其他线程**共享进程**所拥有的全部资源。
- 一个线程可以创建和撤销另一个线程，**同一进程**中的**多个线程**之间可以**并发**执行。
- 由于线程之间的相互制约，致使线程在运行中呈现出**间断性**，也有就绪、阻塞和运行三种基本状态。
- 引入线程后，进程只作为除 CPU 外的**系统资源的分配单元**，而线程则作为**处理机的分配单元**。
- 由于一个进程内部有多个线程，若线程的切换发生在同一个进程内部，则只需要**很少的时空开销**。

> 进程与线程的比较：
>
> - 调度：
> - 在传统的操作系统中，拥有资源和独立调度的基本单位都是进程。
> - 在引入线程的操作系统中，**线程是独立调度的基本单位**，**进程是拥有资源的基本单位**。
> - 在同一进程中，线程的切换不会引起进程切换。
> - **在不同进程中进行线程切换，如从一个进程内的线程切换到另一个进程中的线程时，会引起进程切换**。
>
> > 由于有了线程，线程切换时，有可能会发生进程切换，也有可能不发生进程切换，平摊下来**每次切换所需的开销就变小**了，因此能够让更多的线程参与并发，而不会影响到**响应时间**等问题。

- 拥有资源：

  不论是传统操作系统还是设有线程的操作系统，**进程**都是**拥有资源**（资源分配）的基本单位，而线程不拥有系统资源（只有一点必要的资源），但线程可以访问其隶属进程的系统资源。

  > 若线程也是拥有资源的单位，则切换线程就需要较大的时空开销，线程这个概念的提出就没有意义。

- 并发性：

  在引入线程的操作系统中，不仅进程之间可以并发执行，而且**多个线程之间也可以并发执行**，从而使操作系统具有更好的并发性，提高了系统的吞吐量。

- 系统开销：

  - 由于创建或撤销**进程**时，系统都要为之分配或回收资源，如内存空间、I/O 设备等，因此操作系统所付出的开销远大于创建或撤销线程时的开销。
  - 在进行进程切换时，涉及当前执行进程 CPU 环境的保存及新调度到进程 CPU 环境的设置，而线程切换时**只需保存和设置少量寄存器内容**，开销很小。
  - 由于同一进程内的多个线程**共享**进程的**地址空间**，因此这些**线程之间的同步与通信**非常容易实现，甚至无须操作系统的干预。

- 地址空间和其他资源（如打开的文件）：

  进程的地址空间之间互相独立，同一进程的各线程间共享进程的资源，某进程内的线程对于其他进程不可见。

- 通信方面：

  进程间通信（IPC）需要进程同步和互斥手段的辅助，以保证数据的一致性，而线程间可以直接读/写进程数据段（如全局变量）来进行通信。

- 关系：

  - 线程不像进程那样，不是按照严格的父子层次来组织的。
  - 和一个进程相关的线程组成一个**对等（线程）池**，独立于其他线程创建的线程。
  - **主线程**和其他线程的区别仅在于它总是进程中第一个运行的线程。
  - 对等（线程）池概念的主要影响是，一个线程可以杀死它的任何对等线程，或者等待它的任意对等线程终止，都能读写相同的共享数据。

线程的实现方式（理解）

- 线程的实现分为两类：

> **用户级线程**（User-Level Thread，ULT）和**内核级线程**（Kernel-Level Thread，KLT），内核级线程又称**内核支持的线程**。

- 用户级线程：
  - 有关线程管理（线程的创建、撤销和切换等）的所有工作都由**应用程序**完成，内核意识不到线程的存在。
  - 应用程序可以通过使用线程库设计成**多线程程序**。
  - 应用程序从**单线程（主线程）**开始，在该线程中开始运行，在其运行的任何时刻，可以通过调用线程库中的派生例程创建一个在相同进程中运行的**新线程（对等线程）**。
- 内核级线程：
  - 在内核级线程中，线程管理的所有工作由内核完成，应用程序没有进行线程管理的代码，只有一个到内核级线程的编程接口。
  - 内核为进程及其内部的每个线程维护上下文信息，调度也在内核基于线程架构的基础上完成。
- 组合：
  - 有些系统中使用组合方式的**多线程实现**。线程创建完全在用户空间中完成，线程的调度和同步也在应用程序中进行。
  - 一个应用程序中的多个用户级线程被**映射**到一些（小于等于用户级线程的数目）内核级线程上。

    > 组合模式的多线程模型：
    >
    > - 多对一模型：
    >
    > > 将多个用户级线程映射到一个内核级线程，线程管理在用户空间完成。此模式中，**用户级线程对操作系统不可见**（即**透明**，a 对 b 透明在计算机领域是在 b 的角度 a 不可见的意思）。
    > >
    > > - 优点：
    > >
    > > > 线程管理是在用户空间进行的，效率高，**相同映射**的**用户级线程**之间的切换不需要在内核级中切换。
    > >
    > > - 缺点：
    > >
    > > > - **一个线程在使用内核服务时被阻塞，整个进程都会被阻塞**
    > > > - 任一时间只有一个线程可以访问内核，导致多个线程**不能并行**地运行在多处理机上。

- 一对一模型：

  将每个用户级线程映射到一个内核级线程。

  - 优点：

    当一个线程被阻塞后，允许另一个线程继续执行，所以**并发能力**较强。

  - 缺点：

    每创建一个用户级线程都需要创建一个内核级线程与其对应，这样创建线程的**开销**比较大，会影响到应用程序的性能

- 多对多模型：

  将 n 个用户级线程映射到 m 个内核级线程上，要求 $n\ge m$。集前两者之所长。

Posix 线程

- Posix 线程（Pthreads）是在 C 程序中处理线程的一个标准接口，在所有的 Linux 系统上都可用。
- Pthreads 定义了大约 60 个函数，允许程序**创建、杀死和回收**线程，与对等线程**安全地共享数据**，还可以**通知**对等线程系统状态的变化。
- 创建线程：

```
// 若创建成功则返回 0，若出错则为非零。
int pthread_create(pthread_t *tid,          // 新创建线程的 ID
                   pthread_attr_t *attr,    // 设置新创建线程的属性,为 NULL 就好
                   func *f,                 // 创建线程后将执行的程序（线程例程）
                   void *arg)
// arg 是一个可以传递给开始程序的参数。它必须作为空类型的指针强制转换通过引用传递。如果不传递任何参数，则可以使用 NULL。
```

  新线程可以调用 `pthread_self(void)` 来获得它自己的线程 ID。

  > CSAPP 中使用**线程例程**的概念：
  >
  > - 线程的**代码和本地数据**被封装在一个线程例程（thread routine）中。
  > - 每个线程例程都以一个**通用指针作为输入**，并返回一个通用指针。

- 终止线程：

- 隐式终止：
  - 当顶层的**线程例程返回**时，线程会隐式地终止。
  - 所属进程
- 显式终止：

  通过调用 `pthread_exit` 函数，线程会显式地终止。如果主线程调用 `pthread_exit`，它会**等待**所有其他**对等线程**终止，然后再终止主线程和整个进程，返回值为 `thread_return`。

  ```
  // 无返回值
  void pthread_exit(void *thread_return);
  ```

- 阻塞线程：

  ```
  // 若创建成功则返回 0，若出错则为非零。
  int pthread_join(pthread_t tid, void **thread_return)
  ```

  - `pthread-join` 函数会阻塞当前线程，直到线程 `tid` 终止，将线程例程返回的通用 `(void*)` 指针赋值为 `thread-return` 指向的位置，然后回收已终止线程占用的所有内存资源。
  - 不同于 `wait`，`pthread_join` 只能等待**一个指定的线程**终止。
- 分离线程：
  - 在任何一个时间点上，线程是可结合的（joinable）或者是分离的（detached）。
  - 一个可结合的线程能够**被其他线程收回和杀死**。在**被其他线程回收之前**，它的内存资源（例如栈）是不释放的。
  - 而一个分离的线程是不能被其他线程回收或杀死的。它的内存资源在**它终止时由系统自动释放**。
  - 默认情况下，线程**被创建成可结合的**。为了避免内存泄漏，每个可结合线程都应该要么被其他线程显式地收回，要么通过调用 `pthread detach` 函数被分离。

    ```
    // 若创建成功则返回 0，若出错则为非零。
    int pthread_detach(pthread_t tid);
    ```

线程同步

> 也有叫进程同步，应该没区别吧👶
>
> 整理自 PPT 和《王道》，《王道》题目量巨大

线程内存模型

- 一组**并发线程**运行在一**个进程的上下文**中。
- 每个线程都有它自己**独立**的**线程上下文**，包括**线程 ID、栈、栈指针、程序计数器、条件码和通用目的寄存器值**。
- 每个线程和其他线程一起**共享**进程上下文的**剩余部分**。

  > 共享区域：
  >
  > - 整个用户虚拟地址空间，它是由只读文本（代码）、读/写数据、堆以及所有的共享库代码和数据区域组成的。
  > - 线程也共享相同的打开文件的集合。

- 寄存器是从不共享的，而虚拟内存总是共享的。
- 变量映射到虚拟内存：
  - 全局变量：
    - 全局变量是定义在**函数之外**的变量。
    - 在运行时，虚拟内存的读/写区域**只包含**每个全局变量的**一个实例**，任何线程都可以引用。
  - 本地自动变量：
    - 本地自动变量就是定义在函数内部但是**没有 static 属性**的变量。
    - 在运行时，每个线程的**栈**都包含它自己的所有本地自动变量的实例。即使多个线程执行同一个线程例程时也是如此。
  - 本地静态变量：
    - 本地静态变量是定义在函数内部并**有 static 属性**的变量。
    - 和全局变量一样，虚拟内存的读/写区域只包含在程序中声明的每个本地静态变量的**一个实例**。
  - 共享变量：

    共享变量的一个实例被**一个以上的线程**引用。

    > **本地自动变量也能被共享**，可能被其他线程通过指针间接引用。

同步问题

- 五个阶段：
  - H_{i}：
    > 在循环头部的指令块。
  - L_{i}（load）：
    > 加载共享变量 cnt 到累加寄存器 $\mathrm{%rdx}_{i}$ 的指令，这里 $\mathrm{%rdx}_{i}$ 表示线程 i 中的寄存器 $\mathrm{%rdx}_{i}$ 的值。
  - U_i（update）：
    > 更新（增加）$\mathrm{%rdx}_{i}$。
  - S_{i}（store）：
    > 将 $\mathrm{%rdx}_{i}$ 的更新值存回到共享变量 cnt 的指令。
  - T_{i}：
    > 循环尾部的指令块。
- 进度图：
  - **进度图**将 n 个并发线程的执行模型化为一条 n 维笛卡儿空间中的轨迹线。
  - 每条轴 k 对应于线程 k 的进度。
  - 每个点 $\left(I_{1}, I_{2}, \cdots, I_{n}\right)$ 代表线程 $k(k=1, \cdots, n)$ 已经**完成**了指令 I_{k} 这一状态。
  - 图的**原点**对应于**没有任何线程完成一条指令**的初始状态。
  - 进度图将指令执行模型化为从一种状态到另一种状态的**转换**。
  - 转换被表示为一条从一点到相邻点的**有向边**。向下或者向左移动的转换也是不合法的。
  - 对于线程 i，操作共享变量 cnt 内容的指令 $\left(L_{i}, U_{i}, S_{i}\right)$ 构成了一个（关于共享变量 cnt 的）**临界区**（critical section），这个临界区**不应该和其他进程的临界区交替执行**。
    > **临界区**是指进程中用于访问**临界资源**的**那段代码**。
  - 想要确保每个线程在执行它的临界区中的指令时，拥有对共享变量的**互斥的访问**，这种现象称为互斥。
  - 在进度图中，两个临界区的交集形成的状态空间区域称为**不安全区**。

> 不安全区和与它交界的状态相毗邻，但**并不包括这些状态**。
>
> 例如，状态 $\left(H_{1}, H_{2}\right)$ 和 $\left(S_{1}, U_{2}\right)$ 毗邻不安全区，但是它们并不是不安全区的一部分。
>
> 绕开不安全区的轨迹线叫做安全轨迹线，接触到任何不安全区的轨迹线就叫做不安全轨迹线。
>
>
> <img src="./Operating System.assets/image-20210921211420273.png" alt="image-20210921211420273"/>

- 同步与互斥：
  - 同步（**直接制约**关系）定义：
    - **同步**指为完成某种任务而建立的**两个或多个进程**，这些进程因为需要在某些位置上协调它们的工作次序而**等待、传递**信息所产生的制约关系。
    - 进程间的直接制约关系**源于它们之间的相互合作**。
      > 例如：
      > - 输入进程 A 通过单缓冲向进程 B 提供数据。
      > - 当该缓冲区空时，进程 B 不能获得所需数据而阻塞，一旦进程 A 将数据送入缓冲区，进程 B 就被唤醒。
      > - 当缓冲区满时，进程 A 被阻塞，仅当进程 B 取走缓冲数据时，才唤醒进程 A。
  - 互斥（**间接制约**关系）定义：
    - 当一个进程进入临界区**使用临界资源**时，另一个进程必须**等待**。
    - 当占用临界资源的进程**退出**临界区后，另一进程才允许去访问此临界资源。
    - 进程间的间接制约关系**源于它们之间对临界资源的相互竞争**。
      > 例如：
      > - 在仅有一台打印机的系统中，有两个进程 A 和进程 B。
      > - 若进程 A 需要打印时，系统已将打印机**分配**给进程 B，则进程 A 必须**阻塞**。
      > - 一旦进程 B 将打印机**释放**，系统便将进程 A **唤醒**，并将其由**阻塞态变为就绪态**。
  - 为禁止两个进程**同时进入**临界区，同步机制应遵循以下准则：

- 空闲让进：

  临界区**空闲**时，可以**允许**一个**请求进入临界区**的进程立即**进入**临界区。

- 忙则等待：

  当**已有进程**进入临界区时，其他试图进入临界区的进程必须**等待**。

- 有限等待：

  对请求访问的进程，应保证能在**有限时间**内进入临界区（不一直等待）。

- 让权等待：

  当进程**不能进入**临界区时，应立即**释放**处理器，防止进程忙等待。

信号量

- 信号量 s 是具有非负整数值的全局变量，只能由两种特殊的操作来处理，这两种操作称为 P 和 V :
- $P(s)$:
  - 如果 s 是非零的，那么 P 将 s 减 1，并且**立即返回**。
  - 如果 s 为零，那么就**挂起这个线程**，直到 s 变为非零，而一个 V 操作会**重启**这个线程。
  - 在重启之后，P 操作将 s 减 1，并将控制返回给调用者。
- $V(s)$：
  - V 操作将 s 加 1。
  - 如果有任何线程**阻塞**在 P 操作等待
  - s 变成非零，那么 V 操作会**重启**这些线程中的一个，然后该线程将 s 减 1，完成它的 P 操作。

    > CSAPP 和《操作系统概念》的定义不一样，以后者为准：
    >
    > - CSAPP：
    >
    > V 的定义中没有定义等待线程被重启动的顺序。唯一的要求是 V 必须只能重启一个正在等待的线程。因此，当有多个线程在等待同一个信号量时，不能预测 V 操作要重启哪一个线程。
    >
    > - 《操作系统概念》：
    >
    > V 操作是排队唤醒的。

- Posix 函数：

```
void P(sem_t *s);/* Wrapper function for sem_wait */
void v(sem_t *s);/* Wrapper function for sem_post */
```

  - 信号量提供了一种很方便的方法来确保对共享变量的互斥访问：

    基本思想是将**每个共享变量**（或者一组相关的共享变量）与一个信号量 s（一般**初始为 1**）联系起来，然后用 $P(s)$ 和 V (s) 操作将相应的**临界区包围**起来。

    > 利用信号量不变性：
    >
    > 一个正确初始化的信号量不会出现负值，而出现负值的点在不安全区里面。
    >
    >
    > <img src="./Operating System.assets/image-20210921221906980.png" alt="image-20210921221906980"/>

- 以这种方式来保护共享变量的信号量叫做**二元信号量**，因为它的值总是 0 或者 1。
- 以提供互斥为目的的二元信号量常常也称为**互斥锁**（mutex）。
- 在一个互斥锁上执行 P 操作称为对互斥锁**加锁**，执行 V 操作称为对互斥锁**解锁**。
- 对一个互斥锁加了锁但是还没有解锁的线程称为**占用**这个互斥锁。
- 一个被用作**一组可用资源的计数器**的信号量被称为**计数信号量**。

> 除了提供互斥之外，信号量的另一个重要作用是**调度对共享资源的访问**。
>
> 在这种场景中，一个线程用信号量操作来**通知**另一个线程，程序状态中的**某个条件已经为满足**了。
>
> 两个经典而有用的例子是生产者 - 消费者和读者 - 写者问题

生产者 - 消费者问题

- 生产者和消费者线程共享一个有 **n 个槽的有限缓冲区**。
- **生产者线程**反复地**生成**新的项目，并把它们插入到缓冲区中。
- **消费者线程**不断地从缓冲区中取出这些项目，然后消费（使用）它们。
- 因为插入和取出项目都涉及**更新共享变量**，所以我们必须保证对缓冲区的访问是**互斥**的，不能同时插入和取出。

- 但是只保证互斥访问是不够的，我们还需要**调度对缓冲区的访问**：
  - 如果缓冲区是**满的（没有空的槽位）**，那么**生产者**必须**等待**直到有一个槽位变为可用。
  - 如果缓冲区是**空的（没有可取用的项目）**，那么**消费者**必须**等待**直到有一个项目变为可用。

    > 例子：
    >
    > - 在一个多媒体系统中，生产者编码视频帧，而消费者解码并在屏幕上呈现出来。缓冲区的目的是为了减少视频流的抖动，而这种抖动是由各个帧的编码和解码时与数据相关的差异引起的。缓冲区为生产者提供了一个槽位池，而为消费者提供一个已编码的帧池。
    > - 另一个常见的示例是图形用户接口设计。生产者检测到鼠标和键盘事件，并将它们插入到缓冲区中。消费者以某种基于优先级的方式从缓冲区取出这些事件，并显示在屏幕上。

- 代码实现：
  - 定义结构体：

    ```
    typedef struct {
      int *buf;                /* 缓冲区数组 */
      int n;                   /* 最大槽位数 */
      int front;               /* buf[(front+1)%n] 是第一个项目 */
      int rear;                /* buf[rear%n] 是最后一个项目 */
      sem_t mutex;             /* 二元信号量：确保互斥地访问缓冲区 */
      sem_t slots;             /* 计数信号量：计数缓冲区中可用的槽位 */
      sem_t items;             /* 计数信号量：计数缓冲区中可用的项目 */
    } sbuf_t;
    ```

  - 生产者插入项目到缓冲区：

    ```
    void sbuf_insert(sbuf_t *sp, int item) {
      P(&sp->slots);                          /* 等待可用槽位,与缓冲区加锁顺序不能
    换,否则消费者取不出项目,生产者也只能死等*/
      P(&sp->mutex);                          /* 给缓冲区加锁*/
      sp->buf[(++sp->rear)%(sp->n)] = item; /* 插入项目*/
      V(&sp->mutex);                          /* 解锁缓冲区*/
    ```

```
  V(&sp->items);                        /* 宣告这里有可用的项目 */
}
```

- 消费者从缓冲区取走项目：

```
int sbuf_remove(sbuf_t *sp) {
  int item;
  P(&sp->items);                          /* 等待可用项目*/
  P(&sp->mutex);                          /* 给缓冲区加锁*/
  item = sp->buf[(++sp->front)%(sp->n)];  /* 移除项目 */
  V(&sp->mutex);                          /* 解锁缓冲区*/
  V(&sp->slots);                          /* 宣告这里有可用的槽位 */
  return item;
}
```

读者-写者问题

- 定义：
  - 读者-写者问题是互斥问题的一个概括。
  - 一组**并发的线程**要访问一个**共享对象**，例如一个主存中的数据结构，或者一个磁盘上的数据库。
  - 有些线程只读对象，叫做**读者**；其他的线程只修改对象，叫做**写者**。
  - **写者**必须拥有对对象的**独占**的访问，而**读者可以和无限多个其他的读者共享对象**。
    - 读者和写者是互斥的，写者和写者也是互斥的，而读者和读者不存在互斥问题。
    - 任一写者在完成写操作之前不允许其他读者或写者工作
  - 一般来说，有无限多个并发的读者和写者。

    例子：
    - 一个在线航空预定系统中，允许有无限多客户同时查看座位分配，但是正在预订座位的客户必须拥有对数据库的独占的访问。
    - 在一个多线程缓存 Web 代理中，无限多个线程可以从共享页面缓存中取出已有的页面，但是任何向缓存中写入一个新页面的线程必须拥有独占的访问。
- 读者优先：

- 要求**不要让读者等待**，除非已经把使用对象的权限赋予了一个**写者**。
- 读者不会因为有一个写者在等待而等待。
- 实现：
  - 定义：
    - 信号量 w 控制对访问共享对象的临界区的访问（写者间，读者和写者间互斥访问）。
    - 信号量 **mutex** 保护对**共享量 readcnt** 的访问（读者间的互斥访问），readcnt 统计当前在临界区中的读者数量。
    - 每当一个写者进入临界区时，它对互斥锁 w 加锁，每当它离开临界区时，对 w 解锁。

      > 保证了任意时刻临界区中最多只有一个写者。
    - 只有**第一个进入**临界区的**读者**对 w **加锁**，而只有**最后一个离开**临界区的**读者**对 w **解锁**。
    - 当一个读者进入和离开临界区时，如果还有其他读者在临界区中，那么这个读者会**忽略**互斥锁 w。
    - 这就意味着只要还有一个读者**占用**互斥锁 w，**无限多数量的读者**可以没有障碍地进入临界区。
  - 代码：

```
int readcnt; /* Initially = 0 */
sem_t mutex, w; /* Initially = 1 */
/* 读者 */
void reader(void) {
    while (1) {
        P(&mutex);
        readcnt++;
        if (readcnt == 1) /* 第一个进入 */
            P(&w);
        V(&mutex);
        /* 这里放临界区代码 */
        /* 读 */
        P(&mutex);
        readcnt--;
        if (readcnt == 0) /* 最后一个离开 */
```

```
            V(&w);
        V(&mutex);
    }
}

/* 写者 */
void writer(void) {
    while (1) {
        P(&w);
        /* 这里放临界区代码 */
        /* 写 */
        V(&w);
    }
}
```

- 饥饿：
  - 对这读者 - 写者问题的正确解答可能导致饥饿（starvation），饥饿就是一个**线程无限期地阻塞**，无法进展。
  - 如果有读者不断地到达，写者就可能无限期地等待。
  - 但是从某种意义上说，这种优先级是很弱的，因为一个离开临界区的**写者**可能重启一个在等待的**写者**，而不是一个在等待的读者。其中这种弱优先级会导致一群写者使得一个读者饥饿。

- 写者优先：
  - 要求一旦一个写者准备好可以写，它就会**尽可能快地完成它的写操作**。
  - 在一个写者后到达的**读者**必须**等待**，即使这个写者也是在等待。

基于预线程化的并发服务器

- 定义：
  - 我们为每一个新客户端创建了一个**新线程**，导致开销过大。
  - 一个基于预线程化的服务器试图通过使用如图所示的**生产者 - 消费者**模型来降低这种开销（有一个生产者和多个消费者）。
  - 服务器是由**一个主线程和一组工作者线程**构成的。
  - 主线程不断地接受来自客户端的连接请求，并将得到的**连接描述符**放在一个**有限缓冲区**中。

> - 每一个工作者线程反复地从共享缓冲区中**取出**描述符，为客户端服务，然后**等待**下一个描述符。

image-20210922105410600

- 实现：
  - 在初始化了缓冲区 sbuf 后，主线程创建了**一组工作者线程**。
  - 然后它进入了无限的服务器**循环**，**接受**连接**请求**，并将得到的已**连接描述符**插入到缓冲区 sbuf 中。
  - 每个**工作者线程**的行为都非常简单。它等待直到它能从**缓冲区中取出**一个**已连接描述符**，然后调用 `echo_cnt()` 例程函数**回送**客户端的输入。
  - 函数 `echo_cnt()` 在全局变量 byte_cnt 中记录了从所有客户端接收到的累计字节数，用 mutex 信号量进行互斥保护。

死锁

> 主要来自《王道》

死锁的定义和发生条件

- 信号量引入了一种潜在的令人厌恶的运行时错误，叫做死锁（deadlock），体现为线程因为等待一个永远也不会为真的条件被阻塞了。

  > 死锁是多个进程因**竞争资源**而造成的一种僵局（互相等待），若无外力作用，这些进程都将无法向前推进。
  >
  > 只有对**不可抢占资源（不共享）的竞争**才可能产生死锁，对可抢占资源（贡献）的竞争是不会引起死锁的，如只读文件。

- 程序员使用 P 和 V 操作顺序不当，以至于两个信号量的禁止区域重叠。
- 如果某个执行轨迹线碰巧到达了死锁状态，那么就不可能有进一步的进展了，因为重叠的禁止区域（死锁区域）阻塞了每个合法方向上的进展。

  > 换句话说，程序死锁是因为每个线程都在等待其他线程执行一个根本不可能发生的 V 操作。

image-20210927150402535

- 资源分配图：
  - 通过称为系统资源分配图的有向图可以更精确地描述死锁，包括一个节点集合 V 和一个边集合 $E_{\circ}$

- 节点集合 V 可分成两种类型： $P=\left{P_{1}\right.\left.P_{2}, \cdots, P_{n}\right}$（系统所有**活动进程的集合**）和 $R=\left{R_{1}, R_{2}, \cdots, R_{m}\right}$（系统所有资源类型的集合）。
- 申请边：

  从进程 P_{i} 到资源类型 R_{j} 的有向边记为 $P_{i} \rightarrow R_{j}$，它表示进程 P_{i} 已经**申请**了资源类型 R_j 的一个实例，并且正在**等待**这个资源。
- 分配边：

  从资源类型 R_{j} 到进程 P_{i} 的有向边记为 $R_{j} \rightarrow P_{i}$，它表示资源类型 R_{j} 的一个实例已经**分配**给了进程 P_i。

  > 在图形上，用**圆表示进程** P_{i}，用**矩形表示资源**类型 R。由于资源类型 R_{j} 可能有多个实例，所以矩形内的**点的数量表示实例数量**。
  >
  > 注意申请边只指向矩形 R_{j}，而分配边应指定矩形内的**某个圆点**。
  >
  > 当进程 P_{i} 申请资源类型 R_{j} 的一个实例时，就在资源分配图中加入一条申请边（P 操作阻塞）。
  >
  > 当该申请得到满足时，那么申请边就立即转换成分配边（P 操作之后）。
  >
  > 当进程不再需要访问资源时，它就释放资源，因此就删除了分配边（V 操作之后）。
- 有环是有死锁的**必要条件**：

  image-20210927152310670

  右图因为进程 P_2 可能释放资源类型 R_1 的实例，这个资源可分配给进程 P_1，从而打破环。

- 死锁的四个必要条件：
  - **互斥**：

    在一段时间内某非共享资源**仅为一个进程所占有**。此时若有其他进程请求该资源，则请求进程只能**等待**。
  - **不可抢占**：

    进程所获得的资源在未使用完之前，**不能**被其他进程**强行夺走**，即只能由获得该资源的进程自己来释放（只能是**主动释放**）。
  - **请求和保持条件**：

> 进程已经**保持（已申请到）了至少一个资源**，但又提出了**新的资源请求**，而该资源已被其他进程占有，此时请求进程被阻塞，但对自己已获得的资源**不释放**（保持）。

- **循环等待**：

> 存在一种进程资源的循环等待环，环中每个进程已获得的资源同时被链中下一个进程所请求。即存在一个处于等待态的进程集合 ${P_{1}, P_{2}, \cdots, P_{n}}$，其中 P_{i} 等待的资源被 $P_{i+1}(i=0,1, \cdots, n-1)$ 占有，P_{n} 等待的资源被 P_{0} 占有。

死锁预防

- 思想：

> 破坏产生死锁的 4 个**必要条件**中的一个或几个。

- 破坏互斥条件：

> 若允许系统资源都能共享使用，则系统不会进入死锁状态。
>
> > 有些资源本身就是不能同时访问，如打印机等临界资源只能互斥使用。

- 破坏不可抢占条件：

> 当一个**已保持了某些不可抢占资源**的进程**请求新的资源而得不到满足**时，它必须**释放已经保持的所有资源**，待以后需要时再重新申请（**持有不用会被别的进程抢占**）。
>
> > 缺点：
> >
> > 该策略实现复杂，释放已获得的资源可能造成**前一阶段工作的失效**，**反复地申请和释放资源**会增加系统**开销**，降低系统**吞吐量**。
> >
> > 这种方法常用于状态**易于保存和恢复**的资源，如 **CPU 的寄存器及内存资源**，一般**不能用于打印机**之类的资源

- 破坏请求并保持条件：

> - 采用预先静态分配方法，即进程在运行前一次申请完**它所需要的全部资源**，在它的资源未满足前，不把它投入运行。
> - 一旦投入运行，这些资源就**一直归它所有**，不再提出其他资源请求，这样就可以保证系统不会发生死锁。
> - 另一种方式是**在没有占用资源时，才可申请资源**。在申请新的资源之前，必须释放已分配的所有资源（先释放再申请）。

> 这种方式实现简单，缺点是系统资源被严重**浪费**，利用率低。
>
> 而且还会导致「**饥饿**」现象，由于个别资源长期被其他进程占用时，将致使等待该资源的进程迟迟不能开始运行。

- 破坏循环等待条件：

> - 为了破坏循环等待条件，可采用顺序资源分配法。
> - 首先给系统中的资源编号，规定每个进程必须按**编号递增的顺序请求资源**，同类资源一次申请完。
> - 也就是说，只要进程提出申请分配资源 R，则该进程在以后的资源申请中就只能申请编号大于 R 的资源。
>
> 这种方法存在的问题是：
>
> - 编号必须相对稳定，这就**限制了新类型设备的增加**。
> - 会发生作业使用资源的顺序与系统规定顺序不同的情况，造成**资源的浪费**。
> - 此外，这种按规定次序申请资源的方法，也必然会给用户的**编程**带来麻烦。

死锁避免

在资源的动态分配过程中，用某种方法**防止系统进入不安全状态**，从而避免死锁。

- 避免死锁的方法中，允许进程**动态地申请资源**，但系统在进行资源分配之前，应先计算此次分配的**安全性**。若**此次分配不会导致系统进入不安全状态**，则允许分配，否则让进程等待。

> 安全状态定义：
>
> 系统能按某种进程推进顺序 $\left(P_{1}, P_{2}, \cdots, P_{n}\right)$ 为每个进程 P_{i} 分配其所需的资源，直至满足每个进程对资源的最大需求，使每个进程都可顺序完成。
>
> 此时称 $P_{1}, P_{2}, \cdots, P_{n}$ 为**安全序列**。若系统无法找到一个安全序列，则称系统处于不安全状态。

- 银行家算法是最著名的死锁避免算法，其思想是：

> - 把**操作系统**视为**银行家**，操作系统管理的**资源**相当于银行家管理的**资金**，进程向操作系统**请求分配资源**相当于用户向银行家**贷款**，操作系统按照银行家制定的规则为进程分配资源。
> - 进程运行之前先声明对各种资源的**最大需求量** Max，这一数量不能超过系统资源的总和。

- 当用户**申请**一组资源时，系统应**检测**资源的分配是否仍会使系统处于**安全状态**。如果会，就可分配资源；否则，进程应等待，直到某个其他进程**释放**足够多的资源为止

- 变量定义：

```
int n,m;                //系统中进程总数n和资源种类总数m
int Available[m];       //资源当前可用总量
int Allocation[n][m];   //当前已经分配给每个进程的各种资源数量
int Need[n][m];         //当前每个进程还需分配的各种资源数量
int Work[m];            //当前可分配的资源,可用+可收回的
bool Finish[n];         //进程是否结束
```

- 资源请求算法：

设 $\operatorname{Request}_{i}$ 是进程 P_{i} 的请求向量，$\operatorname{Request}_{i}[j]=K$ 表示进程 P_{i} 需要 j 类资源 K 个。当 P_{i} 发出资源请求后，系统按下述步骤进行检查:

1. 若 $\operatorname{Request}_{i}[j] \leq \operatorname{Need}[i, j]$，则转向步骤 2，否则认为出错，因为它所需要的资源数已超过它所宣布的**最大值**。
2. 若 $\operatorname{Request}_{i}[j] \leq \operatorname{Available} [j]$，则转向步骤 3，否则表示**尚无足够资源**，$P_{{i}}$ 等待。
3. 系统**试探**着把资源分配给进程 $P_{{i}}$，并修改下面数据结构中的数值。

$$
\begin{aligned}
&\text{Available} = \text{Available} - \text{Request}_i[j] \\
&\text{Allocation}[i,j] = \text{Allowcation}[i,j] + \text{Request}_i[j] \\
&\text{Need}[i,j] = \text{Need}[i,j] - \text{Request}_i[j]
\end{aligned}
$$

4. 系统执行**安全性算法**，检查此次资源分配后，系统是否处于**安全状态**。若安全，才正式将资源分配给进程 $P_{{i}}$，以完成本次分配；否则，将本次的试探分配作废，恢复原来的资源分配状态，让进程 $P_{{i}}$ **等待**。

- 安全性算法（时间复杂度 $\mathcal{O}(mn^2)$）：

设置工作向量 $\operatorname{Work} = \operatorname{Available}$，有 m 个元素，表示系统中的剩余可用资源数目。

1. 初始时安全序列为空。
2. 从 Need 矩阵中找出符合下面条件的行：

- 该行对应的进程**不在安全序列**中。
- 该行小于等于 Work 向量（即每一列都小于），找到后，把对应的进程加入安全序列；若找不到，则执行步骤 4。

3. 设步骤 2 中找到的进程 ${P}_{i}$ 进入安全序列后，可顺利执行直至完成，并释放分配给它的资源，此时执行 $\operatorname{Work} = \operatorname{Work} + \operatorname{Allocation}[i]$，其中 $\operatorname{Allocation}[i]$ 表示进程 ${P}_{i}$ 代表的在 $\operatorname{Allocation}[i]$ 矩阵中对应的行，返回步骤 2。
4. 若 3 中安全序列中**已有所有进程**，则系统处于**安全状态**，若未找到满足条件的进程，则系统处于不安全状态。

伪代码：

```
bool Found;
Work = Available;
Finish = false;
while(true){
    Found = false;            //是否为安全序列找到一个进程
    for(i = 1; i <= n; i++)
        if(Finish[i] == false && Need[i] <= Work){      // 这个向量「小于等于」是每个分量都小于等于，O(m)判断
            Work = Work + Allocation[i];
            Finish[i] = true;
            printf("%d->",i); //输出安全序列
            Found = true;
        }
    if(Found == false) break;
}
for(i = 1;i <= n;i++)
    if(Finish[i] == false)
        return "deadlock";     //只要有一个没有分配成功，就是产生了死锁
```

- 缺点：
  - 每个进程进入系统时必须告知**所需资源的最大数量**，对应用程序员要求高。
  - 安全序列寻找算法（安全状态判定算法）计算时间复杂度为 $\mathcal{O}(mn^2)$，过于复杂。
  - 若**每次资源请求**都要调用银行家算法，耗时过大。

> - 当前有资源可用，尽管可能很快就会释放，由于会使整体进程处于不安全状态，而不被分配，致使**资源利用率**大大降低。

死锁的检测及恢复

- 无须采取任何限制性措施，**允许**进程在运行过程中**发生死锁**。通过系统的检测机构及时地**检测出死锁的发生**，然后采取某种措施**解除死锁**（抢占一些进程的资源）。
- 类似银行家算法的死锁检测：

思想：

只要可用资源足够，则分配，发现问题再处理（定时检测或者当发现资源利用率低时检测）。

> $\operatorname{Request}_{i}$ 是进程 P_{i} 的请求向量，$\operatorname{Request}_{i}[j]=K$ 表示进程 P_{i} 需要 j 类资源 K 个，代替原来的 Need 向量。

```
bool Found;
Work = Available;
Finish = false;
if Allocation[i] != 0: Finish[i] = false;
else: Finish[i] = true;        //对于无分配资源的进程,不论其是否获得请求资源,则认为其是完成的。
while(true){
    Found = false;             //是否为安全序列找到一个进程
    for(i = 1; i <= n; i++)
        /*
        *  这里原来是判断 Need[i] <= work
        *  原因是已知 Pi 现在不参与死锁(Request <= Work),可以乐观地认为 Pi 不再需要更多资源就可以完成任务,
        *  它会返回现已分配的所有资源。如果这个假定不正确,那么下次调用死锁检测算法时,就会检测到死锁状态。
        */
        if(Finish[i] == false && Request[i] <= Work){
            Work = Work + Allocation[i];
            Finish[i] = true;
            Found = true;
        }
    if(Found == false) break;
}
```

```
for(i = 1;i <= n;i++)
    if(Finish[i] == false)
        return "deadlock";      //只要有一个没有分配成功，就是产生了死锁
```

- 死锁恢复的方法：
  - 终止进程：
    - **中止所有死锁进程**：
      - 这种方法显然会打破死锁环，但是代价也大。
      - 这些死锁进程可能已计算了较长时间，这些部分计算的结果也要放弃，并且以后可能还要重新计算。
    - 一次**中止一个进程**，直到消除死锁循环为止：

      这种方法的开销会相当大，这是因为每次中止一个进程，都应**调用死锁检测算法**，以确定是否仍有进程处于死锁。
  - 抢占资源：
    - 选择牺牲进程：
      - 应确定**抢占的顺序**使得代价最小。
      - 代价因素：

        死锁进程拥有的资源数量、死锁进程耗时等。
    - 回滚：
      - 必须将进程回滚到某个安全状态（足够打破死锁），以便从该状态重启进程。
      - 要求系统维护有关进程**运行状态的更多信息**。
      - 或者完全回滚，终止进程**重新执行**。
    - 饥饿：
      - 要保证资源不会**总是**从同一个进程中被抢占。
      - 如果一个系统是**基于代价**来选择牺牲进程，那么同一进程可能总是被选为牺牲的，导致这个进程永远不能完成指定任务。应确保一个进程只能**有限次数地被选为牺牲进程**。
      - 最为常用的方法是在**代价因素中加上回滚次数**。

> - 预防死锁和避免死锁都属于**事先预防策略**，预防死锁的限制条件比较**严格**，实现起来较为简单，但往往导致系统的**效率低**，资源利用率低。

> - 避免死锁的限制条件相对**宽松**，资源分配后需要通过算法来判断是否进入不安全状态，实现起来较为**复杂**。

- 死锁忽略：

> 采用最多，以上白学🔮

线程安全

> 整理自 PPT 和 CSAPP

线程安全定义：

线程安全的函数被多个**并发线程反复**地调用时，会一直产生**正确**的结果。

四类线程不安全函数

- 不保护**共享变量**的函数：

> - 利用像 P 和 V 操作这样的同步操作来保护共享的变量可以转变为线程安全函数。
> - 这个方法的优点是在调用程序中**不需要修改**。
> - 缺点是同步操作将**减慢**程序的执行时间。

- 被多个进程调用且**持续保持状态**的函数：

> > 原名为「保持跨越多个调用的状态的函数」，有点拗口
>
> 一个**伪随机数生成器**是这类线程不安全函数的简单例子：
>
> ```
> static unsigned int next = 1;
> /* rand: 返回取值范围为 0..32767 的伪随机数 */
> int rand(void) {
>  next = next * 1103515245 + 12345;
>  return (unsigned int)(next / 65536) % 32768;
> }
> /* srand: set seed for rand() */
> void srand(unsigned int seed) {
>  next = seed;
> }
> ```
>
> - 当调用 `srand()` 为 `rand()` 设置了一个种子后，一个单线程反复地调用 `rand()`，能够预期得到一个可重复的随机数字序列。

- 上面的 `rand()` 函数是线程不安全的，因为当前调用的结果依赖于**前次调用**（可能是其他线程调用的）的中间结果。
- 所以如果**多线程**调用 `rand()` 函数，得到的 `next` 随机数**不均匀甚至是常数**。
- `rand()` 这样的函数变为线程安全的**唯一方式是重写**它，使得它不再使用任何 `static` 变量，而是依靠调用者在参数中传递状态信息。
- 修改后的代码：

```
// 传入当前线程上次调用的得到的随机数变量地址
int rand_r(int *nextp) {
    *nextp = *nextp * 1103515245 + 12345;
    return (unsigned int)(*nextp / 65536) % 32768;
}
```

- 这样做的缺点是：

  修改将是非常麻烦的，而且容易出错。

- 返回指向静态变量的指针的函数：
  - 某些函数，例如 `ctime()` 和 `gethost-byname()`，将计算结果放在一个 `static` 变量中，然后返回一个指向这个变量的指针。
  - 如果我们从并发线程中调用这些函数，那么将可能发生错误，因为正在被一个线程使用的结果会被另一个线程悄悄地**覆盖**了。
  - 有两种方法来处理这类线程不安全函数：
    - 重写函数：

      使得调用者传递存放结果的**变量的地址**。这就消除了所有共享数据，但是它要求程序员能够修改函数的源代码。

    - 使用加锁 - 复制技术：
      - 基本思想是将**线程不安全函数与互斥锁**联系起来。
      - 在每一个**调用位置**，对互斥锁加锁，调用线程不安全函数，将函数返回的结果**复制**到一个**私有的内存位置**，然后对互斥锁解锁。
      - 为了尽可能地减少对调用者的修改，定义一个线程安全的**封装函数**，它执行加锁 - 复制，然后通过**调用这个封装函数**来**取代**所有对线程不安全函数的调用。

```c
/* lock-and-copy version ,取代 ctime 函数*/
char *ctime_ts(const time_t *timep, char *privatep) {
    char *sharedp;
    P(&mutex);
    sharedp = ctime(timep);
    strcpy(privatep, sharedp);
    V(&mutex);
    return privatep;
}
```

- 调用线程不安全函数的函数：
  - 如果函数 f 调用线程不安全函数 g，那么 f 不一定是线程不安全。
  - 如果 g 是第 2 类函数，那么函数 f 也是线程不安全的，只能重写函数 g。
  - 如果 g 是第 1 类或者第 3 类函数，那么只要你用一个互斥锁保护调用位置和任何得到的共享数据，f 仍然可能是线程安全的。

可重入性

- 有一类重要的线程安全函数，叫做**可重入函数**（reentrant function）。
- 其特点在于它们具有这样一种属性：

  当它们被多个线程调用时，**不会引用任何共享数据**。
- 尽管线程安全和可重入有时会（不正确地）被用做同义词，但是它们之间还是有清晰的技术差别。**可重入函数集合**是**线程安全函数**的一个**真子集**。
- 显式可重入函数：
  - 所有的函数参数都是**传值传递**的（即**没有指针**）。
  - 所有的数据引用都是**本地自动栈变量**（即没有引用**静态或全局变量**）。
  - 无论怎么被调用，都可断言它是可重入的。
- 隐式可重入函数：

  允许一些参数是**引用传递**的（即允许传递指针），需要**小心地传递**指向**非共享数据**的指针。
- 线程安全的库函数：
  - 大多数 Linux 函数，包括定义在标准 C 库中的函数都是线程安全的，例如：

    `malloc, free, printf, scanf`

> - 例外：
> > image-20210922143820526
> - 调用这些线程不安全的函数，可以采用加锁复制的方法，缺点有：
> > - 降低了程序的速度。
> > - 某些函数返回指向复杂结构的指针，需要深层复制结构。
> > - 对第 2 类函数如 `rand()` 这样依赖跨越调用的无效。

内存管理

内存管理基本概念

> 整理自《王道》。

程序装入和链接

创建进程首先要将**程序和数据装入内存**。将用户**源程序**变为可在内存中执行的**程序**，通常需要以下几个步骤：

- 编译：
> 由编译程序将用户源代码**编译成若干目标模块**，形成整个程序的完整逻辑地址空间。
- 链接：
> 由链接程序将编译后形成的一组**目标模块**及所需的**库函数**链接在一起，形成一个完整的**装入模块**。
- 装入：
> 由装入程序将**装入模块装入内存**运行。

逻辑地址和物理地址

- 逻辑地址：
> - 编译后，每个目标模块都从 0 号单元开始编址，这称为该目标模块的相对地址（或逻辑地址）。
> - 当**链接程序**将各个模块链接成一个完整的可执行目标程序时，链接程序顺序依次按各个模块的相对地址构成统一的从 0 号单元开始编址的逻辑地址空间。
> - 用户程序和程序员只需知道逻辑地址，而内存管理的具体机制则是完全透明的，只有系统编程人员才会涉及内存管理的具体机制。

> - **不同进程可以有相同的逻辑地址**，因为他们可以映射到主存的不同位置。

- 物理地址：

> - 物理地址空间是指内存中物理单元的集合，它是地址转换的最终地址，进程在运行时执行指令和访问数据，最后都要通过物理地址从主存中存取。
> - 当装入程序将可执行代码**装入内存**时，必须通过地址转换将逻辑地址转换成物理地址（动态重定位貌似不用），这个过程称为**地址重定位**。
>
> > - 逻辑地址空间：
> >
> > > 在实际应用中，将虚拟地址和逻辑地址经常不加区分，通称为逻辑地址。逻辑地址的集合称为逻辑地址空间
> >
> > - 线性地址空间：
> >
> > > **CPU 地址总线可以访问**的所有地址集合称为线性地址空间
> >
> > - 物理地址空间：
> >
> > > **实际存在**的可访问的物理内存地址集合称为物理地址空间

程序的链接的三种方式

- **静态链接**：

> 在程序运行之前，先将各目标模块及它们所需的库函数链接成一个完整的可执行程序，以后**不再拆开**。

- **装入**时动态链接：

> 将用户源程序编译后所得到的一组目标模块，在装入内存时，采用**边装入边链接**的方式。

- **运行**时动态链接：

> 对某些目标模块的链接，是在程序执行中**需要**该目标模块时**才进行**的。其优点是便于修改和更新，便于实现对目标模块的共享。

内存的装入模块在装入内存时，对应有以下三种方式：

- 绝对装入：

> - 在**编译**时，若知道程序将驻留在内存的某个位置，则编译程序将产生绝对地址的目标代码。
> - 绝对装入程序按照装入模块中的地址，将程序和数据装入内存。
> - 由于程序中的**逻辑地址与实际内存地址完全相同**，因此不需对程序和数据的地址进行修改。

    - 绝对装入方式只适用于**单道程序环境**。
- 可重定位装入：
    - 在**多道程序环境**下，多个目标模块的**起始地址**通常都从 0 开始，程序中的其他地址都是**相对于始址**的。
    - 根据内存的当前情况，将装入模块装入**内存的适当位置**。装入时对目标程序中指令和数据的修改过程称为**重定位**，地址变换通常是在装入时一次完成的，所以又称**静态重定位**。
    - 静态重定位的特点是，一个作业装入内存时，必须给它分配要求的全部内存空间，若没有足够的内存，则不能装入该作业。
    - 作业一旦进入内存（一旦装入），整个运行期间就**不能**再在内存中**移动**，也不能再申请内存空间。
- 动态运行时装入（**动态重定位**）：
    - 程序在内存中若发生移动（进程调度），则需要采用动态的装入方式。
    - 装入程序把装入模块装入内存后，并不立即把装入模块中的相对地址转换为绝对地址，而是把这种地址转换**推迟**到程序**真正要执行时才进行**。
    - 因此，装入内存后的所有地址均为相对地址。这种方式需要一个**重定位寄存器**的支持，提供基地址。

      > 每个进程有自己的重定位寄存器，达到保护进程数据的作用。
    - 动态重定位的特点如下：
        - 可以将程序分配到**不连续**的存储区中。
        - 在程序运行之前可以只装入它的**部分代码**即可投入运行，然后在程序运行期间，根据需要动态申请分配内存。
        - 便于**程序段**的共享，可以向用户提供一个**比存储空间大得多的地址空间**。
        - 在**作业执行过程**中进行的，不是装入过程。

内存保护

内存分配前，需要保护操作系统不受用户进程的影响，同时**保护用户进程不受其他用户进程**的影响。内存保护可采取两种方法：

- 在 CPU 中设置一对上、下限寄存器，存放用户作业在主存中的下限和上限地址，每当 CPU 要访问一个地址时，分别和两个寄存器值相比，判断有无越界。
- 采用重定位寄存器（或基址寄存器）和界地址寄存器（又称限长寄存器）来实现这种保护：

> - **重定位**寄存器含**最小**的**物理地址**值，**界地址**寄存器含**逻辑地址的最大值**。
> - 每个**逻辑地址值必须小于界地址寄存器**。
> - 内存管理机构动态地将逻辑地址与界地址寄存器进行**比较**，若**未发生地址越界**，则加上重定位寄存器的值后映射成物理地址，再送交内存单元。
>
> image-20211013235611684

> 内存保护需要**操作系统和硬件**合作完成。

连续分配管理方式

> 整理自《王道》。

- 连续分配方式是指为一个**用户程序**分配一个**连续的内存空间**。
- 碎片：

> **内存**中**剩余的无法使用**的存储空间。

- 外部碎片：

> 随着**进程不断的装入和移出**，对分区不断的分割，使得内存中产生许多特别**小的分区**，它们并不连续可用。

- 内部碎片：

> 对**固定分区**来说，只要分区被分配给某进程使用，其中**并未占用的空间**不能分给其他进程，这些空间称为**内部碎片**。

- 单一连续分配：

> - 内存在此方式下分为系统区和用户区，系统区仅供操作系统使用，通常在低地址部分。
> - 用户区是为用户提供的、除系统区之外的内存空间。
> - 这种方式无须进行内存保护。因为内存中永远只有**一道程序**，因此肯定不会因为访问越界而干扰其他程序。
> - 优点是简单、**无外部碎片**。
> - 缺点是只能用于**单用户、单任务**的操作系统中，有**内部碎片**，存储器的利用率极低。

- 固定分区分配：

> - 是最简单的一种**多道程序**存储管理方式，它**预先**将用户内存空间划分为若干**固定大小**的区域，**每个分区只装入一道作业**。

- 当有**空闲分区**时，便可再从外存的后备作业队列中选择适当大小的作业装入该分区。
- 分区大小相等（等长分区）：

  用于利用一台计算机去控制多个**相同对象**的场合，缺乏灵活性。
- 分区大小不等（变长分区）：

  划分为多个较小的分区、适量的中等分区和少量大分区。
- 特点：

  无外部碎片，但不能实现多进程**共享**一个主存区，存储空间**利用率低**。

- 动态分区分配（**可变分区**分配）：

  > **可变分区**和**变长分区**要区别。

  - 不预先划分内存，而是在**进程装入内存**时，根据进程的大小**动态**建立分区，并使分区的大小正好适合进程的需要（即分区本身没有**内部碎片**）。
  - 系统中分区的大小和数目是可变的。
  - 动态分区在换入换出后会导致内存中出现许多小的内存块（碎片），即**外部碎片**。
  - 动态分区的分配策略：
    - 首次适应（First Fit）

      空闲分区以**地址递增**的次序链接。分配内存时顺序查找，找到能满足要求的第一个空闲分区。
    - 最佳适应（Best Fit）：

      空闲分区按**容量递增**的次序链接，找到第一个能满足要求的空闲分区。
    - 最坏适应（Worst Fit）

      空闲分区以**容量递减**的次序链接，找到第一个能满足要求的空闲分区，即挑选出**最大的分区**。
  - 克服外部碎片可以通过紧凑（合并空闲分区）技术来解决，即操作系统不时地对进程进行移动和整理。但这需要**动态重定位寄存器**的支持，且相对费时。

非连续分配管理方式

基本分段存储管理方式

- 将程序按含义分成若干部分，即分段。**段内要求连续**，**段间不要求连续**，因此整个作业的地址空间是**二维**的
- 汇编时每个段从 0 编址，链接时可不为 0。
- 创建进程（分别载入各个段）时，建立**进程段表**，用于实现从逻辑段到物理内存区的映射：

  image-20211014190118300

  每个进程有自己的进程段表，有多个基址。
- 内存仍用**可变分区**进行管理，载入段时要使用分配算法。
- 从逻辑地址中取出前几位为**段号**，后几位为**段内偏移量**，需要判断：
  - 段号是否存在于段表中
  - **段表项长度**是否大于等于**段内偏移量**，小于会判断为越界
- PC 及数据地址要通过段表算出物理地址，到达内存。
- 进程切换时，**进程段表**也跟着切换。

> 优点：
>
> - 不同的段有不同的含义，可区别对待。
> - 每个段独立编址，编程容易。
>
> 缺点：
>
> 存在**外部碎片**（王道说是**没有内部碎片**的），有较大的资源浪费。
>
> 分段的题目：
>
>
> <img src="./Operating System.assets/image-20211214095011776.png"/>
>
>
>
> <img src="./Operating System.assets/image_71.png"/>

基本分页存储管理方式

- 选择分页的原因：
  - **固定分区**会产生**内部碎片**，**动态分区**会产生**外部碎片**，这两种技术对内存的利用率都比较低。
  - 把主存空间划分为大小相等且固定的块，块相对较**小**，作为主存的基本单位。

- 每个进程也以块为单位进行划分，进程在执行时，以块为单位逐个申请主存中的块空间。
- 分页的方法从形式上看，像等长等长分区的固定分区技术，分页管理不会产生**外部碎片**，块的大小相对分区要小很多，内部碎片大小有上界保证。

- 分页的概念：
  - 页面和页面大小：
    - **进程中的块称为页**，**内存中的块称为页框**（页帧）。
    - 外存也以同样的单位进行划分，直接称为块，大小和页是一样的。
    - 进程在执行时需要申请主存空间，即要为每个页面分配**主存中的可用页框**，这就产生了页和页框的一一对应。
  - 逻辑（虚拟）地址的结构：
    - 地址结构前一部分为页号 P，后一部分为页内偏移量 W。
    - 地址长度为 32 位，其中 $0 \sim 11$ 位为页内地址，即**每页大小**为 $4 \mathrm{~KB}$，$ 12 \sim 31$ 位为页号，地址空间最多允许 2^{20} 页。
    - 地址结构决定了**虚拟内存的寻址空间**有多大。
  - 页表：
    - 为了便于在内存中找到进程的每个页面所对应的物理块，系统为**每个进程建立一张页表**，它记录页面在内存中对应的**物理块号**（页框号或帧号），页表一般存放在**内存**中。
    - 页表是由页表项组成的，注意：
      - 页表项与地址都由两部分构成，第一部分都是**页号**（页表项的页号一般是连续的，所以可以不存页号这一列）。
      - 页表项的第二部分是**物理内存中的块号**，而地址的第二部分是**页内偏移**。
      - **页表项的第二部分与地址的第二部分**共同组成**物理地址**。
      - 页表项结构（PTE）图（上面说的第二部分就是物理页号）：
    - 页表长度是指一个页表中一共有多少页，页表项长度是指页地址占多大的存储空间。
  - 在系统中通常设置一个**页表寄存器**（PTR），存放页表在内存的起始地址 F 和页表长度 L。进程未执行时，页表的始址和长度存放在**进程控制块**中。

- 设页面大小为 L，逻辑地址 A 到物理地址 E 的转换过程：
  1. 计算页号 $P(P=A / L)$ 和页内偏移量 $W(W=A % L)$。
  2. 比较页号 P 和**页表长度**（共有多少页） M，若 $P \geq M$，则产生越界中断，否则继续执行。
     页表中页号 P 对应的页表项地址 $=$ 页表始址 $F~+$ 页号 $P ~\times$ 页表项长度，取出该页表项内容 b，即为**物理块号**。
  3. 计算 $E=b \times L+W$，用得到的**物理地址** E 去访问内存。
- 缺点：
  - 每次访存操作都需要进行**逻辑地址到物理地址**的转换，地址转换过程必须足够快，否则访存速度会降低。
  - 每个进程引入页表，用于存储映射机制，页表不能太大，否则**内存利用率**会降低。
- 具有快表 TLB 的地址变换机构（后面会详细解释）：
  - 在页表的地址变换过程中，若**页表全部放在内存**中，则存取一个数据或一条指令至少要**访问两次内存**：
    - 第一次是**访问页表**，确定所存取的数据或指令的物理地址。
    - 第二次是根据该地址存取数据或指令。
  - 在地址变换机构中增设一个具有并行查找能力的高速缓冲存储器（一组关联快速寄存器组），又称相联存储器（**TLB**），用来存放**当前访问的若干页表项**，以加速地址变换的过程。与此对应，主存中的页表常称为**慢表**。
  - 具体过程：
    - CPU 给出**虚拟地址**后，由硬件进行地址转换，将**虚拟页号**送入 TLB，并将此页号与 TLB 中的所有页号进行比较。
    - 若找到匹配的页号，说明所要访问的页表项在快表中，则直接从中取出该页对应的**页框号**，与页内偏移量拼接形成物理地址。这样，存取数据仅**一次访存**便可实现。
    - 若未找到匹配的页号，则需要**访问主存中的页表**，在读出页表项后，应同时将其**存入快表**，以便后面可能的再次访问。
    - 若快表已满，则必须按照一定的算法对旧的页表项进行替换。
  - 快表的有效性基于著名的**局部性原理**。

二级页表（比较绕）

- 本质上就是在**原有页表结构上再加上一层页表**，顶级页表只有 1 个页面。

> image-20211018160636257

- 一个页表的在内存中占 $4~\mathrm {KB}$，且页面大小也是 $4~\mathrm {KB}$，所以一个页（内存中的块）能放下一个页表。
- 多级页表要达到的目的：

> 在进程执行时，**只需要将顶级页表调入内存**即可，进程的页表和进程本身的页面可在后面的执行中再调入内存（**需要时再调入**）。
>
> > 内存中分成 2^{20} 个页，页表中每个页表项大小为 $4~\mathrm {B}$，如果全部调入内存需要 $4~\mathrm {MB}$ 的大小，而很多页是不需要的。
> >
> > 多级页表就是想让不需要的页表不用调入内存。
> >
> > 如果顶级页表中的一个 PTE 是空的，那么相应的二级页表就根本不会存在，这就是多级页表节约内存的原理。

- 顶级页表正好通过 $1~\mathrm {K}$ 个页表项，得到其他的 $1~\mathrm {K}$ 个次级页表的块号。
- PDE（顶级页表的页表项） 和 PTE 的 $4~\mathrm {B}$ 结构：

```
/* PDE */
|<------ 31~12------>|<------ 11~0 --------->| 比特
                     |b a 9 8 7 6 5 4 3 2 1 0|
|--------------------|-|-|-|-|-|-|-|-|-|-|-|-| 占位
|<-------index------>| AVL |G|P|0|A|P|P|U|R|P| 属性
                            |S|   |C|W|/|/|
                                  |D|T|S|W|

/* PTE */
|<------ 31~12------>|<------ 11~0 --------->| 比特
                     |b a 9 8 7 6 5 4 3 2 1 0|
|--------------------|-|-|-|-|-|-|-|-|-|-|-|-| 占位
|<-------index------>| AVL |G|P|D|A|P|P|U|R|P| 属性
                            |A|   |C|W|/|/|
                            |T|   |D|T|S|W|
```

- 使用二级页表的地址翻译过程（逻辑地址到物理地址）：

> - **逻辑地址**格式：

顶级页号（页目录号） $10~\rm b$	次级页号 $10~\rm b$	页内偏移量 $12~\rm b$

- 回顾只有一级页表的地址翻译过程：

> 设页面大小为 L，逻辑地址 A 到物理地址 E 的转换过程：
>
> 1. 计算页号 $P(P=A / L)$ 和页内偏移量 $W(W=A \% L)$。
> 2. 比较页号 P 和**页表长度** M，若 $P \geq M$，则产生越界中断，否则继续执行。
> 3. 页表中页号 P 对应的页表项地址 $=$ 页表始址 $F~+$ 页号 $P ~\times$ 页表项长度，取出该页表项内容 b，即为**物理块号**。
> 4. 计算 $E=b \times L+W$，用得到的**物理地址** E 去访问内存。

- 在顶级页表中，这个页面大小不是 $4~\rm{KB}$。因为顶级页表的一个页表项对应于一个次级页表所在块，这个次级列表有 $1~\mathrm {K}$ 个页表项，每页是 $4~\rm{KB}$，所以这个次级列表所映射的页面大小之和是 $4~\rm{MB}$。所以在顶级页表中，这个页面大小是 $4~\rm{MB}$。这也对应了逻辑地址格式的顶级页号是 $10~\rm{b}$。
- 在第 3 步根据顶级页号可以找到对应页表项的内容（PDE 的结构中的第 $12-31$ 位），即次级页表所在块号。
- 笔者认为，这时我们不需要用到页内偏移量，可以认为 $W=0$，这样就能找到次级页表的首地址了。
- 在次级页表中，这个地址翻译过程就比较容易理解了，和一级页表翻译过程完全一样，页面大小是 $4~\rm{KB}$，得到次级页号也是 $10~\rm{b}$ 的。

- 多级页表使得地址翻译效率更低，**访问内存次数增加**。

段页式管理方式

- 综合两者优点：

> 页式存储管理能有效地提高**内存利用率**，而分段存储管理能反映**程序的逻辑结构**并有利于段的共享，符合程序员习惯。

- 在段页式系统中，作业的**地址空间首先被分成若干逻辑段**，每段都有自己的段号（面向程序员）。然后将**每段**分成若干**大小固定的页**（面向硬件）。
- 对内存空间的管理仍然和分页存储管理一样，将其分成若干和**页面大小相同的存储块**，对内存的分配以存储块为单位
- 结构图：

- 系统为**每个进程**建立一张**段表**，每个分段有一张**页表**。此外，系统中还应有一个段表寄存器，指出作业的段表始址和段表长度。
- 段表寄存器和页表寄存器的作用都有两个，一是在段表或页表中寻址，二是判断是否越界。

> 在一个进程中，段表只有一个，而页表可能有多个。

- 地址翻译过程：

- 在进行地址翻译时，首先通过段表查到**页表基址**，然后通过页表找到页帧号，最后形成物理地址，进行一次访问实际需要**三次**访问主存，可通过 TLB 加速。

> PPT 上是逻辑地址 $\to$ 线性地址 $\to$ 物理地址，地址格式也不同。

Intel x86 的内存

> 来自 PPT，有的班没讲，可跳过😿

- 实模式：

  逻辑地址与物理地址一致，物理地址 = 段地址 + 偏移地址。

- 保护模式：
  - 内存的管理模式分为**纯段模式**和**段页模式**，没有纯页模式。
  - 段模式是必不可少的，而页模式则是可选的。
  - 如果使用页模式，则是段页式，否则这是纯段模式。
- GDT 和 LDT：
  - 每个**进程**都有自己的**段表**（LDT，存各个进程的段描述符），**操作系统**有自己的段表（GDT，存操作系统的段描述符）
  - GDT 的入口放在 GDTR 寄存器中，GDT 中还记录各个进程的段表 LDT 入口，**当前进程**的 LDT 入口同时放入 **LDTR 寄存器**。
  - GDTR 和 LDTR 寄存器（存放 GDT 和当前进程 LDT 的入口）都有 32 位段表基址（线性空间 $4 \rm ~G$） + 16 位段表长度。
- 寻找**段**对应**内存线性空间**的块位置：
  - 若 $TI = 0$（**段寄存器**的一个标志位）：

    - 先从 GDTR 寄存器中获得 GDT 基址。

- 然后在 GDT 中以段选择器（段寄存器）高 13 位位置索引值得到段描述符。
- 段描述符包含段的**基址、限长**、优先级等各种属性，这就得到了**段的起始地址（基址）**，再以基址加上偏移地址得到线性地址。

> 顺序：
>
> GDTR $\to$ GDT $\to$ 段寄存器索引 $\to$ GDT $\to$ 对应段

- 若 $TI = 1$：

> - 还是先从 GDTR 寄存器中获得 GDT 基址（找 LDT）。
> - 从 LDTR 寄存器中获取 LDT 所在段的位置索引（LDTR 高 13 位），以这个位置索引在 GDT 中得到 LDT 段描述符从而得到 LDT 段基址。
> - 用段选择器高 13 位位置索引值从 LDT 段中得到段描述符。
> - 段描述符包含段的基址、限长、优先级等各种属性，这就得到了段的起始地址（基址），再以基址加上偏移地址得到线性地址。
>
> > 顺序：
> >
> > GDTR $\to$ GDT $\to$ LDTR $\to$ GDT $\to$ LDT $\to$ 段寄存器索引 $\to$ LDT $\to$ 对应段

- 段页模式的地址变换：

> 所有进程共享 $4\rm ~G$ 的线性地址空间，段基址可以重叠，只需要页表不同（CR3 寄存器指向当前进程的页目录）。

虚拟内存

> 整理自《王道》，CSAPP，PPT

虚拟内存的概念

- 之前讨论的内存管理策略的共性和不足：

> - 同时将多个进程保存在内存中，以便允许进行多道程序设计。
> - 一次性：
>
> > 作业必须**一次性全部**装入内存后，才能开始运行。这会导致两种情况：
> >
> > - 当作业很大而不能全部被装入内存时，将使该作业无法运行。

        - 当大量作业要求运行时，由于内存不足以容纳所有作业，只能使**少数作业先运行**，导致多道程序度的下降。
    - 驻留性：
        - 作业被装入内存后，就一直驻留在内存中，其任何部分都**不会被换出**，直至结束。
        - 运行中的进程会因等待 I/O 而被阻塞，可能处于长期等待状态，应当被换出而没有换出。

    > 核心问题是许多在程序运行中**不用或暂时不用的程序**（数据）占据了大量的内存空间，而一些需要运行的作业又无法装入运行，浪费了宝贵的内存资源。

- 关键的局部性原理：
    - 时间局部性：
        - 程序中的某条指令一旦执行，不久后该指令可能再次执行。
        - 某数据被访问过，不久后该数据可能再次被访问。
        - 产生时间局部性的典型原因是程序中存在着大量的**循环**操作。
    - 空间局部性：
        - 一旦程序访问了某个存储单元，在不久后，其**附近的存储单元**也将被访问。
        - 即程序在一段时间内所访问的地址，可能集中在一定的范围之内，因为指令通常是顺序存放、顺序执行的，数据也一般是以**向量、数组、表**等形式**簇聚存储**的。
    - 虚拟内存技术利用局部性原理实现高速缓存。
- 虚拟存储器定义：
    - 基于局部性原理，在程序装入时，将程序的**一部分装入内存**，而将其余部分留在**外存**，就可启动程序执行（部分装入）。
    - 在程序执行过程中，当所访问的信息不在内存时，由操作系统将所需要的部分**调入内存**，然后继续执行程序（按需调页）。
    - 操作系统将内存中暂时不使用的内容**换出**到外存上，从而腾出空间存放将要调入内存的信息（换入换出 or 置换）。
    - 这样，系统好像为用户提供了一个比**实际内存大**得多的存储器，但这个存储器**本身并不存在**，因为称为**虚拟存储器**。
    - 虚拟存储器的大小由计算机的**地址结构**决定，并不是内存和外存的简单相加。

> 虚存的容量要满足以下两个条件:
>
> - 虚存的**实际容量** $\leq$ **内存**容量和**外存**容量之和，这是**硬件的硬性条件**规定的。若虚存的实际容量超过了这个容量，则没有相应的空间来供虚存使用。
> - 虚存的**最大容量** $\leq$ 计算机的**地址位数**能容纳的最大容量。假设地址是 32 位的，按字节编址，一个地址代表 $1 \mathrm{~B}$ 存储空间，则虚存的最大容量 $\leqslant 4 \mathrm{~GB}\left(2^{32} \mathrm{~B}\right)$。这是因为若虚存的最大容量超过 $4 \mathrm{~GB}$，则 32 位的地址，超过限制的空间没有意义。
>
>
> <img src="./Operating System.assets/image-20211214113424964.png"/>

请求分页管理方式

- 请求分页方式概念:
  - 请求分页系统建立在**基本分页系统**基础之上，为了支持虚拟存储器功能而增加了**请求调页功能和页面置换**功能。
  - 需要一定容量的**内存及外存**的计算机系统，还需要有**页表机制**、**缺页中断机构**和**地址变换机构**等硬件支持。
- 页表机制:
  - 请求分页系统在一个作业运行之前，**不要求一次性全部**调入内存，因此在作业的运行过程中，可能出现要访问的页面**不在内存**中（但是在页表中）的情况。
  - 为了支持换入换出功能，要修改页表项的结构，新增一些字段:
- 缺页中断机构:
  - 在请求分页系统中，每当所要访问的页面不在内存中时，便产生一个**缺页中断**，请求操作系统将所缺的页从磁盘调入内存。
  - 此时应将缺页的进程**阻塞**（调页完成**唤醒**），若内存中有空闲块，则分配一个块，将要调入的页装入该块，并修改页表中的相应**页表项**。
  - 若此时内存中没有空闲块，则要**淘汰某页**（若被淘汰页在内存期间被修改过，则要将其写回外存）。
  - 缺页中断作为中断，同样要经历诸如保护 CPU 环境、分析中断原因、转入缺页中断处理程序、恢复 CPU 环境等几个步骤。

- 但与一般的中断相比，它有以下两个明显的区别：
  - 在**指令执行期间**而非一条指令执行完后产生和处理中断信号，属于**内中断**，内外中断的概念在这。
  - **一条指令**在执行期间，可能产生**多次缺页中断**。
- 地址变换机构：

页面置换（淘汰）算法

- 但内存已无空闲空间时，就需要从内存中调出一页程序或数据，送入磁盘的对换区（生磁盘，没有文件系统）。
- 虚拟内存技术调换页面时需要访问**外存**，会导致平均访存时间增加，若使用了不合适的置换算法，则会大大降低系统性能。
- Belady 异常：
  - 内存分配的物理块数增大而页故障数**不减反增**的异常现象。
  - 基于队列实现的置换算法可能发生 Belady 异常，而基于堆栈实现的置换算法不会发生 Belady 异常。
- 算法表格如下：

其他相关概念

- 驻留集定义：
  - 在进程准备执行时，操作系统要决定给特定的进程分配几个页框。
  - 给一个进程分配的**物理页框的集合**就是这个进程的**驻留集**。
  - 需要考虑以下几点：
    - 分配给一个进程的存储量越小，任何时候驻留在主存中的进程数就越多，从而可以提高处理机的时间利用效率。
    - 同时，若一个进程在主存中的页数过少，则尽管有局部性原理，**页错误率**仍然会相对较高。

      > 页错误率 PFF = 页错误/指令执行条数
- 页面分配 + 页面置换策略：
  - **固定**分配**局部置换**：

- 它为每个进程分配一定数目的物理块，在整个运行期间都**不改变**。
- 若进程在运行中发生缺页，则只能从该进程在内存中的页面中选出一页换出，然后调入需要的页面。

> 缺点是难以确定应为每个进程分配的物理块数目：
>
> - 太少会频繁出现**缺页中断**
> - 太多又会使 CPU 和其他**资源利用率**下降

- **可变**分配**全局置换**：
  - 这是最**易于实现**的物理块分配和置换策略，它为系统中的每个进程分配**一定数目**的物理块，操作系统自身也保持一个空闲物理块队列。
  - 当某进程发生缺页时，系统从**空闲物理块队列**（或其他进程物理块）中取出一个物理块分配给该进程，并将欲调入的页装入其中。
  - 这种方法比固定分配局部置换更加**灵活**，可以动态增加进程的物理块，吞吐量大。

    > 缺点是它会**盲目地给进程增加物理块**，进程不能控制它自己的缺页错误率，从而导致系统多道程序的**并发能力**下降。

- **可变**分配**局部**置换：
  - 它为每个进程分配一定数目的物理块，当某个进程发生缺页时，只允许从该进程在内存的页面中选出一页换出，因此**不会影响其他进程的运行**。
  - 若进程在运行中**频繁地缺页**，则系统再为该进程**分配**若干物理块，直至该进程缺页率趋于适当程度。
  - 若进程运行中的缺页率特别低，则可适当**减少**分配给该进程的物理块。

    > 优点是可以**动态**增加进程物理块的数量，还能动态减少进程物理块的数量，在保证进程**不会过多地调页**的同时，也保持了系统的**多道程序并发能力**。
    >
    > 缺点是需要更复杂的实现，也需要更大的开销，但对比频繁地换入/换出所浪费的计算机资源，这种牺牲是值得的。

- **没有**固定分配 + 全局置换策略，因为对各个进程固定分配时，页数是不会变换的。

- 调页策略：

  > 来自《王道》，一般会同时使用这两种策略

  - 预调页策略：

        - 根据**局部性原理**，一次调入若干相邻的页可能会比一次调入一页更高效。
        - 但若调入的一批页面中大多数都未被访问，则又是低效的。
        - 因此，需要采用以**预测**为基础的预调页策略，将预计在不久之后便会被访问的页面预先调入内存。
    - 请求调页策略：
        - 进程在运行中需要访问的页面不在内存而提出请求，由系统将所需页面调入内存。
        - 这种策略调入的页一定会被访问，且这种策略比较易于实现，因此在目前的虚拟存储器中大多采用此策略。
        - 它的缺点是每次**只调入一页**，调入/调出页面数多时会花费过多的 I/O 开销。
- 抖动（颠簸）：
    - 在页面置换过程中，一种最糟糕的情形是，刚换出的页面马上又要换入主存，刚换入的页面马上又要换出主存，这种频繁的页面调度行为称为**抖动或颠簸**。
    - 若一个进程在换页上用的时间多于执行时间，则这个进程就在颠簸。
    - 频繁发生缺页中断（抖动）的主要原因：
        - 某个进程**频繁访问的页面数目**高于**可用的物理页帧数目**（驻留集）。
        - 虚拟内存技术在内存中保留多个进程以提高系统效率。在稳定状态，几乎主存的所有空间都被进程块占据，处理机和操作系统可以直接访问到尽可能多的进程。
        - 但如果管理不当，那么处理机的**大部分时间**都将用于调入调出页面，而不是执行进程的指令，因此会大大降低系统效率。
        - **所有**页面替换策略都可能导致抖动。

        > 因此引入「工作集」的概念，确定进程所需的帧数，防止抖动
- 工作集定义：
    - 工作集是指在**某段时间间隔**内，进程要访问的页面集合。它可以反映了进程在接下来的一段时间内**很有可能会频繁访问**的页面集合
    - 基于**局部性原理**，可以用**最近访问过的页面**来确定工作集。
    - 工作集 W 可由时间 t 和工作集窗口大小 Δ 来确定。

        > 例如，某进程对页面的访问次序如下：
        >
        >
        > <img src="./Operating System.assets/image-20211205202735623.png"/>

> 假设系统为该进程设定的工作集窗口大小 Δ 为 5。在 t_{1} 时刻，进程的工作集为 ${2,3,5}$。在 t_{2} 时刻，进程的工作集为 ${1,2,3,4}$。

- 对于局部性好的程序，**工作集大小**一般会比**工作集窗口** Δ 小很多。
- 抖动问题的工作集视角：

> 若分配给进程的物理块小于**工作集大小**，则该进程就很有可能频繁缺页导致抖动。

- 一般来说分配给进程的物理块数（即驻留集大小）要大于工作集大小。
- 工作集模型的原理：
  - 让操作系统**跟踪每个进程的工作集**，并为进程分配**大于**其工作集的物理块。
  - 落在**工作集内**的页面需要调入**驻留集**中，而落在**工作集外**的页面可从驻留集中**换出**。
  - 若还有空闲物理块，则可以**再调一个进程**到内存以增加多道程序数。
  - 若所有进程的**工作集之和**超过了**可用物理块**数，则操作系统会**暂停**一个进程，将其**页面调出**并将其物理块分配给其他进程，防止出现抖动现象。

地址翻译

- Core i7 地址翻译的概况图：

- 无 TLB 的地址翻译（命中）流程：
  1. 处理器生成一个虚拟地址，并把它传送给 MMU。MMU 生成 **PTE 地址**（页表基址 + 虚拟页号），并从高速缓存/主存请求得到它。

     > 内存管理单元 Memory Management Unit，MMU 是 CPU 芯片上的专用硬件，负责地址翻译。

  2. 高速缓存/主存向 MMU 返回 PTE。
  3. MMU 构造**物理地址**，并把它传送给高速缓存/主存。
  4. 高速缓存/主存返回所请求的数据字给处理器。

     > 需要访问两次（及以上）高速缓存/主存。

- TLB 的结构：
  - 在虚拟页号上的划分：

- 翻译流程（TLB 命中情况）：

  1. CPU 产生一个虚拟地址。
  2. MMU 从 TLB 中取出相应的 PTE。
  3. MMU 将这个虚拟地址翻译成一个物理地址，并且将它发送到高速缓存/主存。
  4. 高速缓存/主存将所请求的数据字返回给 CPU。

  > 只需访问一次高速缓存/主存，即使在多级页表中也是如此，因为 TLB 存放的是最后一级页表的页表项。

- 当 TLB 不命中时，MMU 必须从高速缓存/主存中取出相应的 PTE。新取出的 PTE 存放在 TLB 中，可能会覆盖一个已经存在的条目。

  > 如果 TLB 不命中且缺页，**新换入**的页对应页表项会放入 **TLB** 中，所以缺页时还是只需要访问**两次**内存（第一次发现缺页，第二次用查 TLB 得到的页号访问内存）

- 高速缓存的三种组织方式：

  > 设物理地址位 m 位。

  - 直接映射（直相联）：

标记 $t=m-s-b$ 位	组索引 s 位	块内偏移 b 位

    先通过组索引，找到高速缓存对应的组，每组只有一行（路），匹配它的标记，相等则命中。

  - g 路组相联：

标记 $t+g$ 位	组索引 $s-g$ 位	块内偏移 b 位

    先通过组索引，找到高速缓存对应的组，每组有 g 行（路），匹配 g 个标记，相等则命中。

    > 用中间位作组索引的原因：
    >
    >
    > <img src="./Operating System.assets/image-20211206133823597.png"/>

> 因为**相邻**的内存块（**低位不同而高位相同**）如果可以放在高速缓存**不同的组**，就不容易发生替换，即尽可能同时放入高速缓存。

- 全相联：

标记 $m-b$ 位	块内偏移 b 位

匹配高速缓存**所有标记**，相等则命中。

Linux 虚拟内存系统

> 整理自 PPT 和 CSAPP，《王道》没有，不知道会不会考

- Linux 为**每个进程**维护了一个单独的虚拟地址空间，如下图。

- 内核虚拟内存包含内核中的代码和数据结构，其中某些区域被映射到**所有进程共享**的物理页面。
- 内核为系统中的每个进程维护一个单独的**任务结构**（源代码中的 `task_struct`），如下图。

- 任务结构中的元素包含或者指向**内核运行该进程**所需要的所有信息（例如 PID、指向用户栈的指针、可执行目标文件的名字以及程序计数器）。
- 任务结构中的一个条目指向 `mm_Struct`，它描述了虚拟内存的当前状态。其中含有两个关键字段 `pgd` 和 `mmap`：
  - `pgd` 指向顶级页表（**页全局目录**）的基址，当内核运行这个进程时，就将 `pgd` 存放在 CR3 控制寄存器中。

    > **区域**定义：
    >
    > - Linux 将**虚拟内存**组织成一些**区域**（或段）的集合。
    > - 一个区域就是已经存在着的（已分配的）虚拟内存的**连续片**，这些页是以某种方式相关联的。
    > - 例如，代码段、数据段、堆、共享库段，以及用户栈都是不同的区域。

  - `mmap` 指向一个 `vm_area_structs`（区域结构）的链表，其中每个 `vm_area_structs` 都描述了当前**虚拟地址空间**的一个**区域**：
    - `vm_prot`：

      描述这个区域内所有页的**读写许可**权限。

- `vm_flags`：

  描述这个区域内的页面是与其他进程**共享**的还是这个进程**私有**的。

- 假设 MMU 在试图翻译某个虚拟地址 A 时，触发了一个缺页。这个异常导致控制转移到内核的缺页处理程序，处理程序随后就执行下面的步骤：

  1. 判断虚拟地址 A 是否合法（越界）的，换句话说，A 是否在该进程的某个**区域结构定义的区域**内：

     - 缺页处理程序搜索区域结构的链表，把 A 和每个区域结构中的 vm_start 和 vm_end 做比较。
     - 如果这个指令是不合法的，那么缺页处理程序就触发一个**段错误**，从而终止这个进程。

     > 因为一个进程可以创建任意数量的新虚拟内存区域，所以顺序搜索区域结构的链表花销可能会很大。
     >
     > 因此在实际中，Linux 使用某些数据结构科技，例如在链表中构建了一棵树，并在这棵树上进行查找。

  2. 判断试图进行的内存访问是否**合法**，即进程是否有读、写或者执行这个区域内页面的权限：

     - 例如，这个缺页是不是由一条试图对这个代码段里的**只读页面进行写操作**的存储指令造成的。
     - 若不合法，缺页处理程序会触发一个保护异常，从而终止这个进程。

  3. 其他情况正常换入页面，更新页表。

内存映射

- 内存映射定义：

  Linux 通过将一个**虚拟内存区域**与一个**磁盘上的对象**关联起来，以初始化这个虚拟内存区域的内容，这个过程称为内存映射。

- 映射对象：

  - 文件系统（磁盘）中的普通文件：

    - **一个区域**可以映射到一个**普通磁盘文件**的**连续部分**，例如一个可执行目标文件。
    - 文件区被分成**页大小的片**，每一片包含一个**虚拟页面**的初始内容。
    - 因为按需进行**页面调度**，所以这些虚拟页面没有实际交换进入物理**内存**，直到 CPU **第一次引用**到页面（即发射一个虚拟地址，落在地址空间这个页面的范

围之内）。

- 如果区域比文件区要大，那么就用零来填充这个区域的剩下部分。

- 匿名文件：
  - 一个区域也可以映射到一个**匿名文件**，匿名文件是由**内核**创建的，包含的全是**二进制零**。
  - CPU 第一次引用这样一个区域内的虚拟页面时，内核就在物理内存中找到一个**合适的牺牲页面**，如果该页面被**修改**过，就将这个页面换出来，用二进制零**覆盖牺牲页面并更新页表**，将这个页面标记为是驻留在内存中的。
  - 注意在**磁盘和内存**之间并没有实际的数据传送。因为这个原因，**映射到匿名文件**的区域中的页面有时也叫做**请求二进制零的页**。

- 交换文件：
  - 无论在哪种情况中，一旦一个虚拟页面被**初始化**了，它就在一个由**内核**维护的专门的**交换文件**（交换空间或交换区域）之间换来换去。
  - 交换空间**限制**着当前运行着的**进程**能够**分配的虚拟页面**的总数。

- 共享对象和私有对象：
  - 一个对象可以被映射到虚拟内存的一个区域，要么作为**共享对象**，要么作为**私有对象**。
  - 如果一个进程将一个**共享对象**映射到它的**虚拟地址空间**的一个区域内，那么这个进程对这个区域的任何**写操作**，对于那些也把这个共享对象映射到它们虚拟内存的其他进程而言，也是可见的。而且，这些变化也会反映在磁盘上的**原始对象**中。
  - 而对一个映射到**私有对象**的区域做的改变，对于其他进程来说是**不可见**的，并且进程对这个区域所做的任何写操作都不会反映在**磁盘上的对象中**。
  - 一个映射到**共享对象**的虚拟内存区域叫做**共享区域**，反之称为私有区域。

    > 假设进程 1 将一个**共享对象**映射到它的虚拟内存的一个区域中，如图所示。现在假设进程 2 将同一个共享对象映射到它的地址空间（并**不一定**要和进程 1 在**相同的虚拟地址**处）。
    >
    >
    > <img src="./Operating System.assets/image-20211206204751788.png"/>

- 写时复制：
  - **私有对象**使用一种叫做写时复制的巧妙技术被映射到虚拟内存中。

- 一个私有对象开始生命周期的方式基本上**与共享对象的一样**，在物理内存中**只保存**有私有对象的**一份副本**。
- 如图两个进程将一个**私有对象**映射到它们虚拟内存的不同区域，但是共享这个对象同一个物理副本。
- 对于每个映射私有对象的进程，**相应私有区域的页表条目都被标记为只读**，并且区域结构被标记为**私有的写时复制**。
- 只要没有进程试图**写**它自己的**私有区域**，它们就可以继续共享物理内存中对象的一个**单独副本**。
- 然而，只要有一个进程试图写私有区域内的某个页面，那么这个写操作就会触发一个**保护故障**：
  - 故障处理程序注意到保护异常是由于进程试图**写私有的写时复制区域**中的一个页面而引起的，它就会在**物理内存**中创建这个页面的一个**新副本**，更新**页表条目**指向这个**新的副本**，然后恢复这个页面的**可写权限**。
  - 当故障处理程序返回时，CPU **重新执行这个写操作**，现在在新创建的页面上这个**写操作**就可以**正常执行**了。

  > 通过延迟私有对象中的副本直到最后可能的时刻，写时复制最充分地使用了稀有的物理内存。

- 内存映射的四种类型：

  > PPT 写有但没找到出处，看看就得了。
  >
  > 因为映射对象分为有普通文件和匿名文件，这两种映射又有私有映射和共享映射之分，所以可创建 4 种类型的映射。

  - 普通文件的共享映射：

    多个进程映射到同一个文件，内存修改会被写回到后备文件。用于不同进程间共享内存，以及大文件读写。

  - 普通文件的私有映射：

    多个进程映射到同一个文件，采用**写时拷贝**的方式，一般用在加载共享代码库以及数据段，节省内存。

  - 匿名文件的共享映射：

    内核创建一个初始都是 0 的物理内存区域，然后**父子进程**使用同一个映射，用于关联进程共享内存。

- 匿名文件的私有映射：

  > **内核创建**一个初始都是 0 的物理内存区域，用作进程的私有内存分配，如 `malloc()` 底层分配大块内存。

- 分析 `fork()` 函数：

  - 当 `fork()` 函数被当前进程调用时，内核为新进程创建各种数据结构，并分配给它一个唯一的 PID。
  - 为了给这个新进程创建虚拟内存，它创建了当前进程的 `mm_struct`、区域结构和页表的原样副本。
  - 它将两个进程中的每个页面都标记为**只读**，并将两个进程中的每个区域结构都标记为**私有的写时复制**。
  - 当 `fork()` 在新进程中返回时，新进程现在的虚拟内存刚好和调用 `fork()` 时存在的虚拟内存相同。
  - 当这两个进程中的任一个后来进行写操作时，写时复制机制就会创建新页面，这样就为每个进程保持了私有地址空间的抽象概念。

- 分析函数 `execve("a.out", NULL, NULL);`：

  - 意义：

    > `execve()` 函数在**当前进程**中加载并运行包含在可执行目标文件 `a.out` 中的程序，用 `a.out` 程序有效地替代了当前程序。

  - 加载并运行 `a.out` 需要以下几个步骤：

    - 删除已存在的**用户区域**：

      > 删除当前进程虚拟地址的用户部分中的已存在的区域结构。

    - 映射私有区域：

      > - 为新程序的代码、数据、 `.bss` 和栈区域创建新的区域结构。
      > - 所有这些新的区域都是**私有的写时复制**的。
      > - 代码和数据区域被映射为 `a.out` 文件中的 `.text` 和 `.data` 区。
      > - `.bss` 区域是请求二进制零的，映射到**匿名文件**，其大小包含在 `a.out` 中。
      > - 栈和堆区域也是请求**二进制零**的，初始长度为零。

    - 映射共享区域：

      > 如果 `a.out` 程序与共享对象（或目标）链接，比如标准 C 库 `libc.so`，那么这些对象都是动态链接到这个程序的，然后再映射到用户虚拟地址空间中的共享

区域内。

- 设置程序计数器（PC）：
  - `execve()` 做的最后一件事情就是设置当前进程上下文中的程序计数器，使之指向代码区域的入口点。
  - 下一次调度这个进程时，它将从这个入口点开始执行。Linux 将根据需要换入代码和数据页面。

I/O 管理和存储

> 基本来自《王道》和 PPT

四种 I/O 控制方式

> 典中典，计组学过前三个，第四个要特别注意⚛️

- 程序直接控制方式：
  - 当某进程需要输入/输出数据时，由 CPU 向设备控制器发出一条 I/O 指令启动设备工作（对于输出操作，则 CPU 还要向数据寄存器中存放输出数据）。
  - 在设备输入/输出数据期间，CPU **不断地循环**进行**查询**设备**状态寄存器**的值（检查 I/O 工作是否完成）。
  - 若完成，对输入操作来说 CPU 则从数据寄存器中取出数据，然后进行下一次的输入/输出数据或结束。

    > 由于 CPU 的高速性和 I/O 设备的低速性，致使 CPU 的绝大部分时间都处于等待 I/O 设备的循环测试中，造成了 CPU 资源的极大浪费。
- 中断驱动方式：

  思想是允许 I/O 设备**主动打断 CPU** 的运行并请求服务，从而 CPU 不用等待，使得其向 I/O 控制器发送读命令后可以继续做其他有用的工作。

  > 虽然 CPU 不用等待设备读完，但是数据中的每个字在**磁盘缓存与 I/O 控制器**之间的传输都必须经过 CPU。
  >
  > 因此可以设计有一定处理能力的外围设备，使得这部分工作由它来完成。
- DMA 直接存取方式：

- DMA 方式特点：
  - 基本单位是数据块。
  - 数据在**设备与内存**之间传输，不经手 CPU。
  - 仅在传送一个或多个数据块的**开始和结束**时，才需 CPU 干预，整块数据的传送是在 DMA 控制器的控制下完成的。
- 流程：
  1. 当一个进程要求设备输入数据时，CPU 对 DMA 进行初始化工作，设置各寄存器：
     - 内存地址寄存器：
       > 存放数据的内存起始地址
     - 传送字节数寄存器：
       > 要输入数据的字节数
     - 控制状态寄存器：
       > 控制字（中断允许、DMA 启动位 $=1$）
     - 启动位被置 1，
       > 则启动 DMA 控制器开始进行数据传输。
  2. 该进程放弃 CPU，进入**阻塞**等待状态，等待第一批数据输入完成。进程调度程序调度其他进程运行。
  3. 由 DMA 控制器控制整个数据的传输：
     - 当输入设备将一个数据送入 DMA 控制器的**数据缓冲寄存器**后，DMA 控制器**立即取代 CPU**，接管**数据地址总线**的控制权（CPU 工作周期挪用），将数据送至相应的**内存单元**。
     - DMA 控制器中的传输字节数寄存器计数减 1。
     - 恢复 CPU 对数据地址总线的控制权。
     - 循环直到数据传输完毕。
  4. 当一批数据输入完成，DMA 控制器向 CPU 发出中断信号，请求中断运行进程并转向执行中断处理程序。
  5. 中断程序首先保存被中断进程的现场，唤醒等待输入数据的那个进程，使其变成**就绪状态**，恢复现场，**返回被中断的进程继续执行**。
  6. 当**进程调度**程序调度到要求输入数据的那个进程时，该进程就到指定的内存地址中读取数据进行处理。

> DMA 控制方式与中断驱动方式的主要区别：
>
> - 中断驱动方式在**每个数据需要传输时**中断 CPU，而 DMA 控制方式则是在所要求传送的一批数据全部传送结束时才中断 CPU
> - 中断驱动方式数据传送是在中断处理时由 CPU 控制完成的，而 DMA 控制方式则是在 DMA 控制器的控制下完成的。

- 通道控制方式：
  - I/O 通道是指专门负责输入/输出的**处理机**，是 DMA 方式的发展，是一种硬件技术，进一步减少了 CPU 的干预。
  - 实现 CPU、通道和 I/O 设备三者的并行操作，从而更有效地提高整个系统的资源利用率。
  - CPU 要完成一组相关的读（或写）操作及有关控制时，只需向 I/O 通道发送一条 I/O 指令，以给出其所要执行的**通道程序的首地址**和要访问的 I/O 设备，通道接到该指令后，执行**通道程序**便可完成 CPU 指定的 I/O 任务，数据传送结束时向 CPU 发中断请求。
  - 通道、设备控制器和设备三者之间的控制关系：

    通道控制**设备控制器**，设备控制器控制**设备**工作
  - I/O 通道与一般处理机的区别：

    通道指令的类型单一，没有自己的内存，通道所执行的**通道程序**是放在主机的**内存**中的，也就是说通道与 CPU 共享内存。

    > **I/O 通道与 DMA 方式的区别**：
    >
    > - DMA 方式需要 CPU 来控制传输的数据块大小、传输的内存位置，而通道方式中这些信息是由通道控制的。
    > - 每个 DMA 控制器对应一台设备与内存传递数据，而**一个通道**可以控制**多台设备**与内存的数据交换。
    > - DMA 方式以存储器为核心，中断控制方式以 CPU 为核心。因此 DMA 方式能与 CPU 并行工作。
    > - DMA 方式传输批量的数据，中断控制方式的传输则以字节为单位。

> 模仿《王道》对 4 种 I/O 控制方式的形象总结：
>
> 我 $\to$ CPU
>
> 三食堂商户 $\to$ 外设
>
> 食物 $\to$ 数据
>
> 假设只有读操作，即取餐。
>
> - 程序直接控制：
>
> > 商户**没有渠道通知**我已做好食物，我必须一直站在商户前，等食物做好，这就浪费了我不少的时间😠。
>
> - 中断驱动方式：
>
> > 三食堂冒菜有**通知设备**告诉我可以去取餐了，每当一个食物做好时（假设我买了很多碗食物），设备震动提醒我去拿，与程序直接控制能省去我不少麻烦，但**每做好一个食物就让我去取一次**，仍然比较浪费我的时间😕
>
> - DMA 方式：
>
> > 我花钱雇一位**单线助理**，并向助理**交代好食物所在位置，放在哪里**等信息，把通知设备给了助理，助理负责把食物取回来并放在我希望的位置，取完所有食物后，助理就要给我报告一次（大大节省了我的时间:happy:）。
>
> - 通道方式：
>
> > 助理拥有更高的**自主权**，与 DMA 方式相比，他可以决定把食物存放在哪里，怎么取食物，而不需要我操心。我有可能在多个商户下了单，一位 DMA 类的助理只能负责在**一个商户**取餐，但通道类助理却可以去**多个商户**取餐😁。

缓冲技术

- 使用缓冲区的目的：
  - 缓和 CPU 与 I/O 设备间**速度不匹配**的矛盾。
  - 减少对 CPU 的**中断频率**，放宽对 CPU 中断响应时间的限制。
  - 解决基本数据单元大小（即数据粒度）不匹配的问题。
  - 提高 CPU 与 I/O 设备之间的**并行性**。
- 缓冲技术的实现方法：
  - **硬件**缓冲：
    > 利用**专门的硬件寄存器**作为缓冲区，一般由外设自带的专用寄存器构成。例如 Printer、CD-ROM 等，成本较高。

- **软件**缓冲：

  借助**操作系统**的管理，在**内存**中专门开辟若干单元作为**缓冲区**。

> 下面部分是《王道》的内容，PPT 内容较少，笔者不是很理解其中的过程，看看知道大概意思就好。

- 缓冲区特点：
  - 当缓冲区的数据**非空**时，不能往缓冲区**冲入数据**，只能从缓冲区把数据**传出**。
  - 当缓冲区为**空**时，可以往缓冲区**冲入数据**，但必须把缓冲区**充满**后，才能从缓冲区把数据传出。
- 缓冲技术：
  - 单缓冲：
    - 在**设备和 CPU**之间设置一个缓冲区。
    - 设备和 CPU 交换数据时，先把被**交换数据**写入**缓冲区**，然后需要数据的设备或处理机从缓冲区取走数据。
    - 外设与 CPU 对缓冲区的操作是**串行**的，不能对缓冲区同时读写。
  - 双缓冲：
    - 根据单缓冲的特点，CPU 在传送时间内处于空闲状态，由此引入双缓冲。
    - I/O 设备输入数据时**先装填到缓冲区 1**，在缓冲区 1 **填满后**才开始装填缓冲区 2。
    - 与此同时 CPU 可以从缓冲区 1 中取出数据放入用户进程处理，当缓冲区 1 中的数据处理完后，若缓冲区 2 已**填满**，则 CPU 又从缓冲区 2 中取出数据放入用户进程处理，而 I/O 设备又可以装填缓冲区 1。

      > 必须等缓冲区 2 **充满**才能让 CPU 从缓冲区 2 取出数据。

    - 双缓冲机制提高了处理机和输入设备的并行操作的程度，可以同时读写。

      > - 对于字符设备，若采用行输入方式：
      > - 双缓冲可使用户在输入第一行后，在 CPU **执行第一行**中的命令的同时，用户可**继续向第二缓冲区**输入下一行数据。
      > - 单缓冲情况下则必须**等待一行数据**被 CPU 提取完毕才可输入下一行的数据。

> - 若两台机器之间通信仅配置了单缓冲，则它们在任意时刻都只能实现**单方向**的数据传输。
> - 为了实现双向数据传输，必须在两台机器中都设置**两个缓冲区**，一个用作**发送缓冲区**，另一个用作**接收缓冲区**。
>
>
> <img src="./Operating System.assets/image-20211207111349709.png"/>

- 循环缓冲：
  - 包含多个大小相等的缓冲区，每个缓冲区中有一个**链接指针**指向下一个缓冲区，多个缓冲区构成一个环形链表。
  - 循环缓冲用于输入/输出时，还需要有两个指针 `in` 和 `out` （PPT 的 `Empty` 和 `Full` ）：
    - 对输入而言， `in` 指针指向可以输入数据的第一个**空缓冲区**；
    - 对输出而言， `out` 指针指向可以提取数据的第一个**满缓冲区**
  - 当 `Full = Empty` 时，要么全为空缓冲区，要么全为满缓冲区。
- 缓冲池：
  - 由多个系统公用的缓冲区组成，所有进程均可以共享，由系统管理程序统一管理，负责分配、回收工作。
  - 缓冲区按其使用状况可以形成 3 个队列：
    - **空缓冲**队列
    - **装满输入数据**的缓冲队列（输入队列）
    - **装满输出数据**的缓冲队列（输出队列）。
  - 具有 4 种缓冲区：
    - 用于收容输入数据的工作缓冲区
    - 用于提取输入数据的工作缓冲区
    - 用于收容输出数据的工作缓冲区
    - 用于提取输出数据的工作缓冲区
  - 输入流程：
    - 当**输入进程**需要**去输入数据**时，便从**空缓冲队列**的队首摘下一个空缓冲区，把它作为收容输入工作缓冲区，然后把输入数据输入其中，装满后再

将它挂到**输入队列队尾**。

- 当**计算进程**需要**输入的数据**时，便从**输入队列**的队首取得一个缓冲区作为提取输入工作缓冲区，计算进程从中提取数据，数据用完后再将它挂到**空缓冲队列尾**。

- 输出流程：

  - 当**计算进程**需要**去输出数据**时，便从**空缓冲队列**的队首取得一个空缓冲区，作为收容输出工作缓冲区，当其中装满输出数据后，再将它挂到**输出队列队尾**。
  - 当要输出时，**输出进程**从**输出队列**的队首中取得一个装满输出数据的缓冲区，作为提取输出工作缓冲区，当数据提取完后，再将它挂到**空缓冲队列的队尾**。

  > 总结：
  >
  > - 输入：
  >
  > 输入进程 $\to$ 空缓冲队列队首 $\to$ 输入队列队尾 $\to$ 计算进程 $\to$ 输入队列队首 $\to$ 空缓冲队列队尾
  >
  > - 输出：
  >
  > 计算进程 $\to$ 空缓冲队列队首 $\to$ 输出队列队尾 $\to$ 输出进程 $\to$ 输出队列队首 $\to$ 空缓冲队列队尾

- PPT 上的补充：

  - Linux 系统为了提高读写磁盘的效率，会先将数据放在一块 buffer 中。
  - 在**写磁盘**时并不是立即将数据**写到磁盘**中，而是先写入这块 buffer 中。
  - 如果 buffer 未写到磁盘，重启系统后，就可能造成数据丢失。
  - `sync` 命令用来 flush 文件系统 buffer，这样数据才会真正的写到磁盘中，并且 buffer 才能够释放出来。
  - `sync` 命令会强制将数据写入磁盘中，并释放该数据对应的 buffer，所以常常会在**写磁盘后**输入 `sync` 命令来将数据真正的写入磁盘。
  - 如果不去手动的输入 `sync` 命令来真正的去写磁盘，Linux 系统也会有两种写磁盘的时机：

    - kflush 内核线程周期性的去写磁盘。
    - buffer 已满不得不写。

SPOOLing 技术（假脱机技术）

- 全称 Simultaneous Peripheral Operation On Line **外部设备同时联机操作**，又称假脱机操作。
- 思想和目的：
  - 以**空间换时间**
  - 将**独占设备**改造成**共享设备**，是一种虚拟设备技术
  - 缓和 CPU 的高速性与 I/O 设备低速性之间的**矛盾**，将低速 I/O 设备上的数据传送到**高速磁盘**。

    > 磁盘是一种高速设备，在与内存交换数据的速度上优于打印机、键盘、鼠标等中低速设备。
- 输入井和输出井：
  - 输入井和输出井是指在**磁盘**上开辟出的两个存储区域。
  - **输入井**模拟脱机输入时的磁盘，用于收容 **I/O 设备**输入的数据。
  - **输出井**模拟脱机输出时的磁盘，用于收容**用户程序**的输出数据。

    > PPT 上写的「输出井用于收容 I/O 设备的输出数据」，输入/输出如果是对设备来说的，这种说法可以接受。
    >
    > 但是一般输入/输出都是对计算机说的，例如鼠标作为输入设备，打印机作为输出设备。
- 输入缓冲区和输出缓冲区：
  - 输入缓冲区和输出缓冲区是在**内存**中开辟的两个缓冲区。
  - **输入**缓冲区用于**暂存由输入设备**送来的数据，以后再传送到**输入井**。
  - **输出**缓冲区用于**暂存从输出井**送来的数据，以后再传送到**输出设备**。
- 输入进程和输出进程：
  - 输入流程：
    - **输入进程**模拟脱机输入时的外围控制机，将用户要求的数据从输入机通过**输入缓冲区**再送到**输入井**。
    - 当 CPU 需要**输入的数据**时，直接将数据从**输入井**读入内存。
  - 输出流程：
    - **输出进程**模拟脱机输出时的外围控制机，把用户要求输出的数据先从**内存**送到**输出井**。

> - 待输出设备空闲时，再将输出井中的数据经过**输出缓冲区**送到**输出设备**。
>
> > 如输出设备是打印机，则把要打印的数据送输出井，CPU 不用等待打印机取完数据再做其他事情。
> >
> > 并且向磁盘输出数据的速度比向打印机输出数据的速度快。

- 独占设备变共享设备的实现过程：

> 截获向某独享设备输出的数据，暂时保存到**内存缓冲区或磁盘文件**中，并进行**排队**，之后逐个输出到外设上。
>
> > SPOOLing 技术实现了**虚拟设备**功能。
> >
> > 多个进程同时使用一独享设备，而对每一进程而言，都认为自己独占这一设备，从而实现了设备的**虚拟分配**，该设备是**逻辑**上的设备。

磁盘

> 概念较多且杂👨‍🎨，整理自 PPT 和《王道》，参考了一些网上资料

磁盘概念

在读/写操作期间，磁头固定，磁盘在下面高速旋转。

- 磁盘盘面上的数据存储在一组同心圆中，称为磁道。每个磁道与磁头一样宽，一个盘面有上千个磁道。
- 磁道又划分为几百个扇区，每个扇区固定存储大小（通常为 $512\rm~B$），一个扇区也称为一个盘块。
- 由于扇区按固定圆心角度划分（**不同半径的磁道扇区数相同**），所以密度从最外道向里道增加，磁盘的存储能力受限于最内道的最大记录密度。
- **扇区**就是磁盘可寻址的最小存储单位，磁盘地址用「柱面号 盘面号 扇区号（或块号）」表示。
- 物理盘以**扇区**为单位进行编址，它是**硬盘读写的基本单位**。
- 在磁盘上进行一次读写操作花费的时间由寻道时间、延迟时间和传输时间决定：

> - 寻道时间是将磁头移动到指定磁道所需要的时间，
> - 旋转时间是磁头定位到某一磁道的扇区（块号）所需要的时间，
> - 传输时间是从磁盘读出或向磁盘写入数据所经历的时间。
>
> > 一般来说，寻道时间因为要移动磁臂，所以占用时间最长。

>
> <img src="./Operating System.assets/image-20211214211950929.png"/>

磁盘调度算法

> 应该必考

- First Come First Severed，FCFS 先来先服务磁盘调度：
  - 按请求磁道的先后顺序
  - 保证公平性，效率不高
- Shortest-Seek-Time First，SSTF 最短寻道时间优先磁盘调度：
  - 运用贪心思想，先访问与当前磁头所在磁道距离最近的磁道，以便使**每次的寻找时间最短**。
  - 性能好，存在**饥饿**问题，有些远磁道总是不能被访问。
- SCAN 扫描/电梯算法磁盘调度：
  - 在磁头**当前移动方向**上选择与当前磁头所在磁道距离最近的请求作为下一次服务的对象。
  - 实际上就是在**最短寻找时间优先算法**的基础上规定了磁头运动的方向，SCAN = SSTF + 中途不回折，**一直移动到 0 或最大磁道才回头**。
  - 不存在饥饿，但是会导致**延迟不均**，不利于远离磁头一端的访问请求。
- C-SCAN 循环扫描/电梯算法磁盘调度：
  - 在 SCAN 的基础上规定磁头**只在一个方向上提供服务**，**回返时**直接快速移动至另一端而**不服务任何请求**，下图就是只在向外的方向服务。
  - **两端请求**都能很快处理，解决 SCAN 算法对两端磁道请求的不公平的问题。
- C-LOOK 磁盘调度：
  - C-SCAN 基础上，磁头移动只需要到达**最远端的一个请求**即可返回，**不需要**到达磁盘**端点**。
  - 更加合理，综合了以上的优点。

磁盘管理

- 磁盘初始化：
  - 一个新的磁盘只是一个含有磁性记录材料的空白盘。
  - 在磁盘能存储数据之前，它必须**分成扇区**以便磁盘控制器能进行读和写操作，这个过程称为低级格式化（**物理格式化**）。
  - 低级格式化使每个扇区的数据结构通常由头、数据区域（通常为 $512\rm~B$ 大小）和尾部组成，头部和尾部包含了一些磁盘控制器所使用的信息。
  - 为了使用磁盘存储**文件**，操作系统还需要将自己的数据结构记录在磁盘上：
    - 第一步将磁盘分为由一个或多个柱面组成的**分区**（即我们熟悉的 C 盘、D 盘等形式的分区）。
    - 第二步对物理分区进行**逻辑格式化**（创建文件系统），操作系统将初始的**文件系统**数据结构存储到磁盘上，这些数据结构包括空闲和已分配的空间及一个初始为空的目录。
- 扇区编号：

  > 与 PPT 的计算不同，一般题目会给出从 0 还是 1 开编址

  - $\rm CHS$ 模式：
    - 采用 3D 参数：
      - 磁柱面数（Cylinders）
      - 磁头数（Heads），即盘面数
      - 扇区数（Sectors per track）
    - 这里扇区其实是从 1 开始编址的（出于某些历史原因），盘面和柱面则是从 0 开始，即第一个扇区编号为 $(0,0,1)$。
  - $\rm LBA$ 编号：
    - 是逻辑 $\rm CHS$ 寻址的简单换算，变为线性寻址。
    - 假设当前 $\rm CHS$ 编号为 (c,h,s) 公式为 $\mathrm{LBA}=c\times \mathrm{H}\times \mathrm{S}+h\times \mathrm{S} + (s-1)$

      > $\rm S$ 为一个磁道所含扇区数，$\rm H$ 为一个柱面所含盘面数

    - 维基百科为证：

- 磁盘往往不是严格按需读取，而是每次都会预读，即使只需要一个字节，磁盘也会从这个位置开始，顺序向后读取**一定扇区长度**的数据放入内存。

  > 这样做的理论依据是**局部性原理**：
  >
  > 当一个数据被用到时，其附近的数据也通常会马上被使用。

磁盘布局

- 物理盘以扇区为单位进行编址，它是**硬盘读写**的**基本单位**。
- 一块**硬盘**从逻辑上可以理解为连续的**一维扇区**序列。
- 整个硬盘的第 1 个扇区存储着**主引导记录**（MBR）：

  - 引导可执行代码
  - 硬盘基本分区表，最多包含 4 个**基本分区**位置信息

  > - 计算机启动时需要运行一个初始化程序（**自举程序**），它初始化 CPU、寄存器、设备控制器和内存等，接着启动操作系统。
  > - 为此，该自举程序应找到磁盘上的操作系统内核，装入内存，并转到起始地址，从而开始操作系统的运行。
  > - 自举程序通常保存在 ROM 中，为了避免改变自举代码而需要改变 ROM 硬件的问题，因此只在 ROM 中保留**很小的自举装入程序**，将完整功能的自举程序保存在磁盘的**启动块**（启动块位于磁盘的 MBR 中）上

- 除了第 1 个扇区之外，其余扇区可以划分为至多 4 个基本分区。
- 每个分区的**第 1 个扇区**预留，可以作为**引导扇区**。
- 每个分区除第 1 个扇区外的其他部分还可以看做一个**硬盘**（同样具有类似「主引导记录」的**可扩展分区引导记录**），继续递归分区
- 可扩展分区：

  > 可以继续划分成「分区」的硬盘分区，其划分可以无限制进行，直到硬盘划分完成为止。

- 可扩展**分区引导记录**：

  > 可扩展分区中第 2 个扇区中的内容（第 1 个为引导扇区）。

- 引导分区：

  > 标记有**可引导标记**的硬盘分区，这种分区有引导扇区和引导文件。

SSD 固态硬盘

> 来自 PPT，有的班没讲，可以跳过

- 由 NAND FLASH 作为**存储介质**，特点有：
  - **顺序**访问比随机访问快
  - 随机**写**较慢（擦除块需要较长的时间，修改一页需要将块中所有页**复制**到新的块中）
- 与机械硬盘对比：
  - 优点：
    > 没有移动部件，读写更快、能耗更低、更结实。
  - 缺点：
    - 会磨损
    - 只能把存储单元从 1 变为 0，导致写数据无法直接覆盖，即不能在已有数据的存储单元上直接修改。
      > - 对于**机械硬盘**来说，就像是写**铅笔字**，可以说是**无限次的写入次数**
      > - 对于**固态硬盘**来说，就像是写**钢笔字**，固态硬盘中需要把有效数据抄到草稿纸（缓存）上，将原先的一整页撕掉（标记为无效），然后再把新的数据和草稿上的有效数据放回去（空的新页）。
- FTL，Flash Translation Layer 闪存翻译层模块：
  - SSD 内部软件的一个主要模块，用于兼容运行在**机械硬盘上的文件系统**。
  - FTL 要实现作为中间层，对外提供的是块接口的功能。作为主机端，所看到的 SSD 是一个和机械硬盘一样的**块设备**。
  - FTL 从主机文件系统接收块级请求，经过 FTL 的处理，产生 flash 的各种控制命令。
  - 由三部分组成：
    - 地址映射：
      > 对于文件系统而言，它所看到的存储空间是一个线性的连续空间，对 SSD 的请求是**逻辑扇区号**，FTL 要转换成 NAND FLASH 中的**物理页号**。
    - 损耗平衡：
      - Flash 中每个块都有一定的擦写次数限制。故不能让某一个块被写次数较多，而其他块被写的次数较少，即尽量平均。
      - **动态**损耗平衡：

在请求到达时，选取擦除次数较少的块作为请求的物理地址。

- **静态**损耗平衡：
  - 在运行一段时间后，有些块存放的数据一直没有更新（**冷数据**），而有些块的数据经常性的更新（**热数据**）。
  - 那些存放冷数据的块的擦除次数远小于存放热数据的块。
  - 将**冷数据**从原块取出，**存放**在擦除次数过多的块；原来存放冷数据的块被释放出来，接受热数据的擦写。
- 垃圾回收：
  - SSD 在使用过程中，对文件持续反复的修改，块中会产生大量失效页，需要把有效的数据从块上搬走，然后擦除该块用来接收新的数据。
  - 垃圾回收需要考虑到冷热数据、磨损平衡和读写延迟。
  - 热数据会被经常更新，在垃圾回收流程中应该避免回收热数据。
  - 如果垃圾回收流程控制不好，会造成读写延迟，时而流畅，时而卡顿。

文件系统

> 整理自《王道》和 PPT，杂乱无章，体现在《王道》的题目基本和 PPT 不相干，笔记也不太好做🤕
>
> 文件系统的定义：
>
> - 操作系统中负责管理和存储文件信息的软件机构称为文件管理系统，简称文件系统。
> - 文件系统由三部分组成：
> 与文件管理有关的软件、被管理文件及实施文件管理所需的数据结构。

文件的物理结构

> 逻辑结构是用户组织数据的结构形式，物理结构是操作系统组织物理存储块的结构形式

- 物理盘块连续分配：
  - 连续分配要求**每个文件**在磁盘上占有**一组连续的块**，需要记录起始盘块和所占盘块个数。
  - 连续的磁盘地址定义了磁盘上的一个线性排序，使访问磁盘时需要的**寻道数和寻道时间**最小。
  - 优点：

简单快速，支持顺序访问和直接访问。

- 缺点：
  - 类似**内存管理**的**连续分配方式**中的**动态分区分配**。
  - 文件长度不宜**动态增加**，因为一个文件**末尾后的盘块**可能已分配给其他文件，一旦需要增加，就需要**大量移动盘块。**
  - 反复增删文件产生大量**外部碎片**。

- 链式（链接）分配：
  - 采取离散分配的方式，**消除了外部碎片**，因此显著提高了磁盘空间的利用率。
  - 无须事先知道文件的大小，可**动态增长**，对文件的**增、删、改**也非常方便。
  - 分类：
    - 隐式链接：
      - 每个文件对应一个磁盘块的**链表**。
      - 磁盘块分布在磁盘的任何地方，除最后一个盘块外，每个盘块都有指向**下一个盘块的指针**，这些指针对用户是透明的。
      - **目录**包括文件**第一块的指针**和**最后一块的指针**（用于追加文件）。
      - 缺点：
        - 无法**直接访问**盘块，只能通过指针顺序访问文件。
        - 盘块**指针**会消耗一定的存储**空间**。
        - **稳定性不高**，系统在运行过程中由于软件或硬件错误导致链表中的指针丢失或损坏，会导致文件数据的丢失。
    - 显式链接：
      - 把用于链接文件各物理块的指针，从每个物理块的块末尾中提取出来，**显式**地存放在内存的一张**链接表**中。
      - 该表在整个磁盘中仅设置一张，称为**文件分配表**（File AllocationTable，FAT）
      - 每个**表项**中存放对应块的**下一块链接指针**，即下一个盘块号。
      - 文件的第一个盘块号记录在**目录**中，后续的盘块可通过查 FAT 找到。
      - FAT 表在系统启动时就会被读入内存，因此查找 FAT 的过程是在**内存**中进行的，因此不仅显著地提高了**检索速度**，而且明显减少了**访问磁盘**的次数。

> 🔔
>
> 据说显式链接是可以实现随机访问的，因为要随机读一个文件的一个块，先在内存中读 FAT 找块所在位置，不用按照隐式链接一样多次读磁盘，只需读一次磁盘就行。但是 PPT 没有讲这部分内容，所以遇到题目链式分配默认指隐式链接。

- 索引分配：
  - 每个文件都有其**索引块**，这是一个磁盘块地址的数组，由**文件的所有的盘块号**构成。
  - 连续和链式分配的有效折衷，支持**直接（随机）访问**，没有**外部碎片**。
  - 缺点是由于索引块的分配，增加了系统存储空间的开销。
  - 采用多种方式解决问题：
    - 链接方案：

      一个索引块通常为一个磁盘块，因此它本身能直接读写。为了处理大文件，可以将多个索引块链接起来。
    - 多级索引：

      多级索引使第一层索引块指向第二层的索引块，第二层索引块再指向文件块，以此类推更高级的索引。
    - **混合索引**：

      将多种索引分配方式相结合的分配方式。例如，系统既采用直接地址，又采用单级索引分配方式或两级索引分配方式。

      > 💃 作业有出，必考，下面是《王道》中一个使用混合索引的例子：
      >
      >
      > <img src="./Operating System.assets/image-20211207214656200.png"/>
      >
      > - 直接地址：
      > - 为了提高对文件的检索速度，在索引结点中可设置 10 个直接地址项，这里每项中所存放的是该文件数据所在盘块的盘块号。
      > - 假如每个盘块的大小为 $4\rm~KB$，当文件不大于 $40\rm~KB$ 时，便可直接从索引结点中读出该文件的全部盘块号。
      > - 一次间接地址：

- 对于大、中型文件，只采用直接地址并不现实。可再利用索引结点中的地址项来提供一次间接地址。
- 一次间址块也就是**索引块**，系统将分配给文件的多个盘块号记入其中。
- 在一次间址块中可存放 1024 个盘块号，因而允许文件长达 $1024\times4\rm~KB=4\rm~MB$。

- 多次间接地址：
  - 当文件长度大于 $\rm 4~MB + 40~KB$ （一次间接地址与 10 个直接地址项)时，系统还须采用二次间接地址分配方式。
  - 系统此时在二次间接地址块中记入所有一次间接地址块的盘号。
  - 在采用二次间接地址方式时，文件的最大长度可达 $4\rm~GB$。
  - 同理，三次间接地址允许的文件最大长度可达 $4\rm~TB$。

目录结构

- 在**检索**目录文件的过程中，只用到了**文件名**，仅当找到**与查找文件名匹配**的**目录项**时，才需要从该目录项中读出该文件的物理地址。
- 也就是说，在检索目录时，文件的**其他描述信息**不会用到，也**不需要**调入内存。
- 有的系统（如 UNIX，以及实验要用的 Ext2）采用了**文件名和文件描述信息**分离的方法，文件描述信息单独形成一个称为**索引结点**的数据结构，简称 i 结点。在文件目录中的每个**目录项**仅由文件名和**指向该文件所对应的 i 结点的指针**构成。

文件系统的实现

- 文件系统作用：
  - PPT 中的描述：
    - 将盘块转换成文件集，方便用户访问。
    - 类比将 CPU 资源和地址空间封装成进程。
  - 《王道》中的描述：
    - 对于**用户**而言，文件系统最主要的功能是实现对**文件的基本操作**，让用户可以**按名存储和查找文件**，组织成合适的结构，并应当具有基本的文件共享和文件保护功能。

> - 对于**操作系统**本身而言，文件系统还需要管理**与磁盘的信息交换**，完成文件**逻辑结构和物理结构**上的变换，组织文件在磁盘上的存放，采取好的**文件排放顺序**和**磁盘调度方法**以提升整个系统的性能。

- 文件系统的数据结构：
  - 文件控制块（FCB）：
    - 存放控制文件需要的各种信息的数据结构，以实现「按名存取」。FCB 的有序集合称为**文件目录**，一个 FCB 就是一个文件**目录项**。
    - FCB 主要包含以下信息：
      - 基本信息，如文件名、文件的物理位置、文件的逻辑结构、文件的物理结构等。
      - 存取控制信息，如文件存取权限等。
      - 使用信息，如文件建立时间、修改时间等。
    - UFS（UNIX 文件系统）称之为**索引结点**（inode）。
  - 引导控制块：

    包括从该卷引导**操作系统**的信息（可为空，没有操作系统），通常为卷的第一块，UFS 称之为**引导块**。
  - 卷控制块：

    包括卷（或分区）的详细信息，如分区的块数、块的大小、空闲块的数量和指针、空闲 FCB 的数量和指针。UFS 称之为**超级块**。
  - 每个文件系统的目录结构：

    用来组织文件，UFS 中包含文件名和相关的索引结点。
- 分区空闲盘块的管理方法：
  - **空闲位图**（位向量）：

    每一位对应一个盘块是否被占用，但可能很大，放不进内存。
  - **空闲链表**：

    空闲块链接成链表。分配**一个（或少量的）**空闲盘块速度很快，虽然分配多个块很慢，但一般每次申请都只需一个或几个空闲块。
- 磁盘缓存：

  在内存中缓存磁盘上的少部分盘块，下图来自 PPT，实在不知道咋总结了😢。

文件系统的保护

- 文件用来存放用户的信息，用户应该能控制对文件的访问，如只允许读。这些读/写/执行（r/w/x）权限放在文件头。
- RAID 冗余保护：
  - 从外部看：

    > RAID 像一个磁盘。

  - 在内部：

    > RAID 由多个磁盘、内存（包括易失性和非易失性）以及一个或多个处理器来管理系统。

  - 性能：

    > 可以并行使用多个磁盘，大大加快 I/O 时间。

  - RAID 1：

    > 直接磁盘镜像。
    >
    > > 数据分散：
    > >
    > > 在多个磁盘上分散数据（也叫数据条带），存取数据时，并行读取多个磁盘，大大提高传输率。
    > >
    > > - 位级分散：
    > >
    > > > 在多个磁盘上分散**每个字节的各个位**，如有 4 个磁盘，每个字节的 i 和 $4+i$ 位，分散在第 i 个磁盘上。
    > >
    > > - 块级分散：
    > >
    > > > **一个文件的块**可分散在多个磁盘上，对于 n 个磁盘，一个文件的块 i，可以存在磁盘 $(i % n) +1$ 上。

  - RAID 0：

    > 采用数据条带（数据分散）技术，增加传输效率，但没有任何校验信息。

  - RAID 2 和 RAID 3：

    > - 都是将数据分散（条块化）分布于不同的硬盘上，采用**位级分散**。
    > - RAID 2 采用**汉明码**检验和纠错，需要额外的多个硬盘进行存储。
    > - RAID 3 使用专用校验盘（一个）存**奇偶校验值**。

  - RAID 4：

- 原理同 RAID 3，只是采用的是**块级分散**。
- 对于每次写操作，都需要重新计算奇偶变化，因此 RAID 4 要执行**四次磁盘** I/O：
  - 两次读取（读目标磁盘块，读奇偶校验块）
  - 两次写入（写目标磁盘块，写奇偶校验块）
- 数据盘可以并行读写，但奇偶检验盘只有一个，**不能并行**操作，导致**对不同盘的写**受到检验盘的瓶颈限制。

- RAID 5：
  - 采用**块级分散**，校验块分布在多个盘上，**消除**校验盘瓶颈。
  - 写操作可以同时发生在完全不同的磁盘上，而 RAID 4 不行。
  - 一块盘掉线的情况下，RAID 5 照常工作。

EXT2 文件系统

- 整个硬盘的第 1 个扇区存储着**主引导记录**（MBR），其余扇区被划分到 4 个基本分区。
- 分区中第一个扇区为**引导扇区**（Boot Sector），引导扇区的大小是确定的，用来存储磁盘分区信息和启动信息，任何文件系统都不能使用引导扇区，引导扇区之后才是 EXT2 文件系统的开始。
- 除了引导扇区之外，EXT2 磁盘分区被顺序划分为若干个**磁盘块组**（Block Group），并且从 0 开始编号。
- 每个块组按照相同的方式组织，每个块组包含等量的物理块（即块组**大小是相同**的；物理分区最后一个块组可能小些），在块组的**数据块**中存储文件或目录。
- 磁盘**块组**中的磁盘块按顺序被组织成：
  - 一个用作**超级块**（Super Block）的磁盘块：
    - 存放了文件系统超级块的一个拷贝，多个块组中的超级块形成**冗余**。
    - 在某个或少数几个超级块被破坏时，可用于**恢复**被破坏的超级块信息。
    - 系统运行期间，把超级块复制到系统缓冲区内，只需把**块组 0 的超级块**读入**内存**，其它块组的超级块做为**备份**。
  - 多个记录**块组描述符**（GDT）的磁盘块：

> - 块组描述符用来描述一个**磁盘块组**的相关信息，整个分区**分成多少个块组**就对应有多少个**块组描述符**。
> - 存放于超级块所在块的下一个块中。
> - 保存在这个块组的 inode 表起始位置，数据块起始位置，空闲的 inode 和数据块个数等信息。
> - 类似超级块有多份拷贝。
- 一个记录数据块位图的磁盘块
- 一个记录索引结点位图的磁盘块
- 多个用作索引结点表的磁盘块（采用**混合索引**）
- 多个用作数据块的磁盘块

- 存储在磁盘上的文件与用户所「看到」的文件有所不同：
  - 用户感觉，文件在逻辑上是连续的
  - 在磁盘上，存储文件数据的磁盘块可能分散在磁盘各处

Windows 的 FAT 文件系统实现

- FAT 卷结构示意图：

引导区	文件分配表 FAT 1	文件分配表 FAT 2	根目录	其他目录和文件

  维基百科图：

  > 下面无情复制 PPT，未知是否会考到🖍️

- 引导区：

- 文件分配表 FAT 1：
  - 功能：
    > 记录和描述整个卷使用情况。
  - FAT 文件系统格式信息：
    - FAT 12/FAT 16/FAT 32
    - 数字越大，FAT 卷容量与簇大小的越大

> - 卷上**每一簇**对应 FAT 中**一项**，记录**该簇使用情况**，包含簇地址号和使用标志信息，使用标志信息 = 0，则该簇空闲未用
> - 每个目录/文件的文件分配链（簇链）：
>
> **链尾（结束）标志信息**为 $\rm0xFFF/0xFFFF/0xFFFFFFFF$

- 文件分配表 FAT 2：
> - 是 FAT 1 的镜像**备份**。
> - 因为文件分配表对卷非常重要，它的内容破坏会导致部分文件无法访问，甚至导致整卷瘫痪。
- 后面两个非常恶心，就不做笔记了🤦，直接进入**文件访问过程**：

>
>
> 目的：
>
> 读出文件 `\aa\bb\ccc.dat` 的内容：
>
> > 查找 `aa` 过程用 <font color=#549c05> 绿色 </font> 框线标出，查找 `bb` 用 <font color=#0f4fad> 蓝色 </font> 框线标出，查找 `ccc.dat` 用 <font color=#c1cc03> 黄色 </font> 框线标出。
>
> 1. 「根目录」不用通过 FAT 表找到，应该是可以知道其簇位置的。然后查簇中的目录项，找到含目录名 = `aa` 的目录项。
> 2. `aa` 目录项中查出该**目录文件**的**首簇号** = 2。
> 3. 查 FAT 1 中以第 2 簇为头的文件分配**簇链**（$2\to6$），检索簇号为 2 和 6 的簇内容，找出含目录名 = `bb` 的目录项。
> 4. 从 `bb` 目录项中查出该目录文件的首簇号 = 7。
> 5. 查 FAT 1 中以第 7 簇为头的文件分配簇链（只有 7），检索簇号为 7 的簇内容，找出含文件名 = `ccc.dat` 的目录项。
> 6. 从 `ccc.dat` 目录项中查出该文件的首簇号 = 4。
> 7. 查 FAT 1 中以第 4 簇为头的文件分配簇链（$4\to8\to5$），读出簇号为 4，8 和 5 的簇内容，即得到了文件 `ccc.dat` 的内容。